Abstract

We study the Online Bipartite Matching, a problem well-studied due to it's usefulness in modelling for various businesses, in various related models with the primary goal of assessing what model modifications are required for deterministic algorithms to break the 0.5 competitiveness barrier. The problem was initially introduced by Karp et al and has since been analyzed under various assumptions with restrictions on the adversary such as forcing a random arrival order being more successful than those that try to empower the algorithm with more information or capabilities like the ability to wait before matching. We also present a competitive algorithm for Online Bipartite Matching with Corrections

Chapter 1

Introduction

We study various problems related to Online Bipartite Matching, where an algorithm must construct the largest matching possible where one anticlique is always visible and the other becomes visible over time and has to be handled immediately. This is a well-studied field as a bipartite graph can represent an 'assignable' relation, the maximum matching - maximal assignment and the popular online version of this problem - an abstraction of assignment whose one side arrives over time which is a good description of many business problems. Moreover, algorithms for this problem can often be generalized to the AdWords problem, where advertisers may bid for impressions within individual budgets and the goal of the algorithm is to maximize revenue, which is the precise system used by advertisement companies today. As such, this problem has a known optimal algorithm, Ranking, with a competitiveness of $1 - \frac{1}{e}$ There is, however, an issue regarding using Online Bipartite Matching as a model for real problems as it is often too restrictive and algorithms based on it aren't as efficient as they could be. In some problems the algorithm may be able to wait, or make small corrections which could improve the size of the found matching but cannot be reflected in the Online Bipartite Matching problem. Many variations of this problem have therefore been analyzed, from restricting the adversary by imposing random vertex arrival or even arrival of vertices from a known distribution to enhancing the algorithm with In this work we analyze these findings in terms of enhancing the algorithm or restricting the adversary, as well as introduce our own analysis, regarding matching with delay and an matching with corrections

Contents

Chapter 2

Preliminaries

$[n] = \{1, \ldots, n\}$, for $f : X \to Y, X' \subseteq X$, $f(X')$ denotes $\{y \in Y | \exists x \in X' : f(x') = y\}$. For vector p, p_i denotes the i-th element of p. Little o complexity notation means strictly slower growth, defined as $f(x) = o(g(x)) \iff \lim_{n\to\infty} \frac{f(x)}{g(x)} = 0$

Graphs A graph $G = (V, E)$ is a structure composed of a set of vertices V and a set edges E which are unordered pairs of vertices. A degree of $v \in V$ $\deg(v) = |\{u \in V | (u, v) \in E\}|$ is the amount of edges from v. A full graph, where $E = \{(v, u) | v, u \in V\}$ contains all possible edges is also called a clique. An anticlique is a graph where $E = \emptyset$ is empty. A bipartite graph $G = (U, V, E)$ is comprised of two anticliques, ie $E \cap V \times V = \emptyset$, $E \cap U \times U = \emptyset$.

Matching A matching $M \subseteq E$ is a subset of edges that have no common endpoints $\nexists e_1, e_2 : \exists v \in V : v \in e_1 \wedge v \in e_2$. A matching M is maximum when it isn't a subset of another matching. A matching is maximal, denoted $M^\star$, if there is no larger matching. A perfect matching is a matching with $\frac{|V|}{2}$ edges, ie covering all vertices. We write $M(v) = u$ to denote that $(u, v) \in M$. We write u is matched to v to denote the same. An alternating path is a sequence of adjacent vertices in a graph $(v_1, \ldots, v_k)$, $\forall i \in [k-1] : (v_i, v_{i+1}) \in E$ where edges alternate between being in a matching and not $\forall i \in [k-2] : [(v_i, v_{i+1}) \in M \implies (v_{i+1}, v_{i+2}) \notin M] \wedge [(v_i, v_{i+1}) \notin M \implies (v_{i+1}, v_{i+2}) \in M]$ that is maximal in length. An augmenting path is an alternating path where the first and last edge are not in the matching. A matching is maximal iff there are no augmenting paths.

Online Algorithms An online algorithm is an algorithm in which the data isn't entirely known, but rather arrives over time and the algorithm makes decisions (usually irrevocably) without the knowledge of the remaining data. An online algorithm can be considered a greedy optimization algorithm. For the purposes of analysis, an online problem is can be considered a game between tha algorithm and a player who is aware of the algorithm. An adversary can be either oblivious,

if it doesn't know the state of the algorithm (valuations of internal variables), or adaptive if it does. Competitiveness is a way of measuring the performance of online algorithms and is a lower/upper bound (depending on whether the goal is to maximize/minimize a parameter). In the case of online algorithms for bipartite matching competitiveness of algorithm ALG is $\inf_{G \text{ graph}} \inf_{\text{arrival order}} \frac{\text{ALG}(G)}{\text{OPT}(G)}$ a guarantee on the minimum size of the matching found by ALG as a fraction of the largest matching OPT. A $\frac{\text{ONLINE}}{\text{OFFLINE}}$ notation is sometimes used as an offline algorithm can find the maximal matching. In the case that an algorithm is randomized, it's competitiveness is $\inf_{G \text{ graph}} \inf_{\text{arrival order}} \frac{\mathbb{E}[\text{ALG}(G)]}{\text{OPT}(G)}$. An algorithm's performance on a graph is the infimum over data order of the expected size of found matching. An algorithm has an asymptotic performance is $\lim_{n\to\infty}$ of performance on graphs of size n. We may refer to the idea of 'time' for an online algorithm, every 1 unit of time from time of 0 a packet of data arrives

Linear Programming Linear Programming is the optimization of a linear function $f(x) = \sum c_i x_i = c^T x$ under linear constraints $Ax \leq b, x \geq 0$. For vectors of equal length x, y we write $x \leq y$ when $\forall_i x_i \leq y_i$. Every linear program (the primal program) has a dual program associated with it, defined as $b^T y$ under $A^T y \geq c, y \geq 0$. If the primal program is a minimization of f then the dual is a maximization and vice versa. A feasible solution is a value of x that satisfies program constraints. An important property of the dual program is Weak Duality: for all feasible primal, dual solutions x, y, $c^T x \leq b^T y$. This allows for the dual program to be used as a bound for the primal. This technique can be used to bound competitiveness of algorithms if the algorithms can be expressed as a linear program

Chapter 3

Semi-Online Model

In this section we will introduce the classical semi-online model introduced by [KVV90]

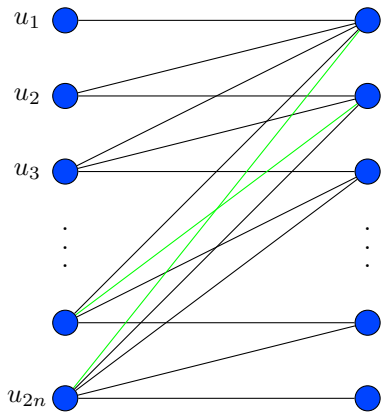
Definition 3.0.1 (Semi-Online Model)**.** *In this model the two anticliques of the bipartite graph $G = (U, V, E)$ are divided into offline and online agents. Offline agents U are vertices that belong to one anticlique and are always visible. Online agents V are the vertices from the other anticlique and arrive in the online fashion. When an online agent arrives edges incident to it are revealed. It has to be either discarded or matched to an offline agent and that match cannot change later. The adversary is oblivious*

For the purposes of minimizing repetition we state that properties of this model, mainly: U being always revealed, the matching being instant and irrevocable, the adversary being oblivious are present in all subsequent model unless specified otherwise

This structure, where one anticlique is always known, will be retained by all models we will present. It is not a necessity, a Fully-Online model where both anticliques arrive in an online fashion exists, but it out of the scope of this work as it is known to be strictly harder than the Semi-Online model.

The adversary is oblivious as an adaptive adversary is trivial.

Remark 3.0.1. *No algorithm has a competitiveness asymptotically higher than $\frac{1}{2}$ against an adaptive adversary*



Proof. Begin with $2n$ offline agents V and $k = 0, X_0 = \emptyset$.
Adversary repeats the following n times: reveals an agent v connected to $U \backslash X_k$.
$k = k + 1$, for some unmatched $u + k \in U; U_{k+1} = U_k \cup \{u_k\}$.
Then adversary reveals n agents denoted V' connected to already matched offline agents. The ultimate size of the matching is n.

In the final graph, all matched vertices V' are connected to all online agents and all unmatched v_i agents are connected to all u_j such that $j \geq i$. Clearly, it is possible to match v_i to u_i and V' to U', forming a matching of size $2n$

□

Remark 3.0.2. *In the semi-online model, any deterministic algorithm has a competitiveness no greater than $\frac{1}{2}$, there is a deterministic algorithm with competitiveness of $\frac{1}{2}$*

Algorithm 1 Naive

on Arrival of vertex v: match it with any offline agent if available

Proof. Trivially the competitiveness cannot be greater than $\frac{1}{2}$ as against a deterministic algorithm there is no difference between an adaptive and an oblivious adversary.

To prove that there is a deterministic algorithm with competitiveness of $\frac{1}{2}$ consider the a naive algorithm.

for every $(u, v) \in E(G)$ either u or v is matched. Therefore at least half of all vertices that could be matched are matched, therefore the competitiveness isn't lower than $\frac{1}{2}$ □

The main discovery of [KVV90] is the following algorithm

as well as the following theorems

Algorithm 2 Ranking

on Initialization: create a random permutation of the offline agents σ
on Arrival of vertex v: match it with the first available offline neighbor according to σ

Theorem 1. *Ranking has asymptotic competitiveness of* $1 - \frac{1}{e}$

Theorem 2. *No algorithm has asymptotic competitiveness higher than* $1 - \frac{1}{e}$

3.1 Ranking Competitiveness

In this section we present a simple proof for Theorem 1 by [BM08]. Many proofs of this theorem use linear programming, but this one uses no special tools.

We treat σ, a permutation of U representing a ranking, as an ordered list or a function $[|U|] \to U$ meaning $\sigma(i)$ is the vertex at rank i in σ and $\sigma^{-1}(u)$ is the rank of v in σ

Lemma 3.1.1. *Competitiveness is determined by graphs with a perfect matching*

Proof. Let G be a bipartite graph without a perfect matching, with orderings of offline agents σ and of online agents π. Consider G' which is a copy of G without a vertex x and orderings π' induced by $V \backslash \{x\}$.

Observe that $M = \text{Ranking}(G, \pi, \sigma)$ being the matching found by the Ranking algorithm on G with ranking π and arrival order σ is either identical to $M' = \text{Ranking}(G', \pi', \sigma')$ or has one more vertex pair. This is apparent since either:

1. x isn't in M,
2. $x' = M(x)$ isn't matched in M', or
3. x' is matched to y in M'

In the case of 3 there is an alternating path of edges in m' and those that are in M but not in M'. When this path ends in the same partition as x then M is larger than M' by 1

This also leads to the conclusion that if the respective matching sizes are different, they differ by a single alternating path.

Now consider the maximal matching of G and the effect the removal of the vertices outside of it will have on the competitiveness. The size of the maximal matching will (of course) not change, while the Ranking's solution decreases by at most 1 with each removal, implying that this process of reducing G to G_p with a perfect matching didn't increase the competitiveness, meaning that analysis of graphs without perfect matchings is redundant. □

We will therefore perform analysis on graphs are assumed to have a perfect matchings.

Lemma 3.1.2. *Let v online vertex,$u = M^\star(v)$, σ permutation.*
If u isn't matched by Ranking(σ) *then v is matched by* Ranking(σ) *to u' with a lower rank*

Proof. Trivial since u, v are connected, therefore for u to remain unmatched v must've been matched to another vertex, which must've in turn had a lower rank than u □

Lemma 3.1.3. *Let v online vertex,$u = M^\star(v)$, σ permutation, σ_i permutation obtained by removing v and inserting it into σ at the i-th place.*
If u isn't matched by Ranking(σ) *then for every i* Ranking(σ_i) *matches v with a vertex u_i such that $\sigma_i(v_i) \leq \sigma(v) = t$*

Proof. $M =$ Ranking(σ),$M_i =$ Ranking(σ_i).
Symmetrically to Lemma 3.1.1 M, M_i are either identical or differ by a single alternating path. By repeatedly applying the logic from the proof of Lemma 3.1.2 see that offline vertices in that path are of increasing rank in σ_i. Therefore $u_i = M_i(v)$,$u' = M(v)$,$\sigma_i^{-1}(u_i) \leq \sigma_i^{-1}(u')$
Using Lemma 3.1.2 we get $\sigma^{-1}(u') < t$ and since $|\sigma_i^{-1}(u') - \sigma^{-1}(u')| \leq 1$ we receive $\sigma_i(u_i)^{-1} < 1 + t$ equivalent to $\sigma_i(u_i)^{-1} \leq t$ on integers □

The final lemma required to complete the proof uses this result directly.

Lemma 3.1.4. *For random σ, the probability that the offline vertex of rank t is matched by* Ranking(σ) *denoted p_t fulfills the following:* $1 - p_t \leq \frac{1}{n} \sum_{1 \leq s \leq t} p_s$

Proof. Let σ be a random permutation of offline vertices. Define σ' with the following random process: choose an offline vertex randomly with equal probability, take it out of σ and insert it into the t-th rank. Let u be the vertex of rank $t, v = M^*(u), R_{t-1}$ the online vertices matched by the algorithm to offline vertices of rank $\leq t - 1$
probability that u is not matched is $1 - p_t$. Expected cardinality of R_{t-1} is $\sum_{1 \leq s \leq t-1} p_s$.
Applying Lemma 3.1.3 with $\sigma := \sigma', i$ chosen such that $\sigma'_i = \sigma$: if u isn't matched by Ranking(σ') then v is matched by Ranking(σ) to a vertex u' such that $\sigma^{-1}(u) \leq t$ implying $u \in R_t$
Conditional on σ, $u \in R_t$ has a probability of $\frac{|R_t|}{n}$ since they are independent.
We now have two random events E, F that fulfill the condition that $E \implies F$.
It is obvious to see that $\Pr[E] \leq \Pr[F]$
The lemma follows

□

With Lemma 3.1.4 the Theorem can be proved directly.
G has a perfect matching implies that competitiveness is inf $\frac{1}{n} \sum_{1 \leq s \leq n} p_s$.
Let $S_t = \sum_{1 \leq s \leq t} p_s$. Lemma 3.1.4 can be rewritten as $S_t(1 + \frac{1}{n}) \geq 1 + S_{t-1}$ and the competitiveness ratio as $\frac{1}{n} S_n$.
Infimum occurs therefore when all inequalities are tight, resulting in $S_t = \sum_{1 \leq s \leq t} (1 - \frac{1}{n+1})^s$ for all t, implying that competitiveness is at least $\frac{1}{n} \sum_{1 \leq s \leq n} (1 - \frac{1}{n+1})^s = 1 - (1 - \frac{1}{n+1})^n \xrightarrow[n \to \infty]{} 1 - \frac{1}{e}$

Thus the theorem is proven

3.2 Ranking Optimality

In this section we present a proof for Ranking Optimality (Theorem 2) by [KVV90]. The main vehicle for this is Yao's lemma with a construction of a class of matching instances. Using the lemma, performance of any algorithm is bounded by the performance of a deterministic greedy algorithm on a randomized instance, which we will prove is equivalent to the performance of RGA on a specific instance, which we will approximate using differential equations

Algorithm 3 RGA

on Arrival of vertex v: match it with a random available offline neighbor

Definition 3.2.1. *bipartite online matching instance (n, σ) where σ is a permutation of $[n]$ is constructed as follows:*

- *offline vertices are u_i for $i \in [n]$*
- *online vertices are v_j for $j \in [n]$*
- *u_i, v_j are connected iff $\sigma(i) \leq j$*

The collection of all $n!$ (n, σ) instances is I. $U(I)$ is the uniform distribution over I. $I^\star$ is $(n, (1, \ldots, n))$, the original instance without permutation

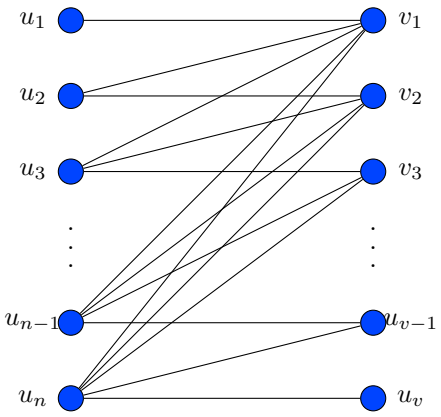


Figure 3.1: Instance $I^\star$

Instance $I = (n, \sigma)$ is the instance $I^\star$ after offline agent order is reshuffled according to σ notice that perfect matching is achieved only if u_i is matched with $v_{\sigma(i)}$ for all $i \in [n]$

Lemma 3.2.1. *Let* DGA *be a greedy deterministic algorithm.*
The expected performance of DGA *on a random instance from* $U(M)$ *bounds the expected performance of* RGA *on* $I^\star$

$$\mathop{\mathbb{E}}_{i \sim U(I)}[|\mathrm{DGA}(i)|] = \mathbb{E}[\mathrm{RGA}(I^\star)]$$

Proof. To prove the lemma we first need to prove two claims:

Claim 3.2.1 (1)**.** *for* DGA *on* (n, σ) *as well as* RGA *on* $I^\star$ *if at time* t *there are* k *eligible offline vertices then they are equally likely to be any set of* k *vertices among the last* $n - t + 1$

Claim 3.2.2 (2)**.** *for each* k *the probability that there are* k *eligible offline vertices at time* t *is the same for* DGA *on* (n, σ) *as well as* RGA *on* $I^\star$

Claim 1 is proven with induction over time - tha amount of online vertices that have already arrived. Note that induction base at step 0 is trivial since all offline vertices are eligible.
Note that a vertex is available if it can still be matched ie isn't matched and isn't unmatchable

Proof. of Claim 1

for RGA.

$$\Pr[S \text{ available at time } t{+}1] = \sum_{v \notin S} \Pr[S \cup \{v\} \text{ available at time } t] \Pr[v \text{ picked at time } t]$$

and since probabilities at time t are equal and the probability of choosing all v is either equal or 0 then the condition is maintained
for DGA, since it is deterministic and the algorithm can't read labels, then the subsequent matching is determined by the current available set, which is uniformly random, implying the next one is also uniformly random. □

Proof. of Claim 2
It is a direct consequence of Claim 1. If the offline vertices eligible for matching are uniformly distributed in both cases, then the probability that a vertex becomes unmatchable is also equal for both algorithms, and that probability is the probability that the amount oa eligible vertices decreases by 2 instead of 1 □

The lemma follows directly from Claim 2 since the size of the matching is equal to the amount of steps required for there to be no eligible vertices □

By using Yao's lemma we receive the following: for any algorithm ALG

$$|\mathrm{ALG}(M)| := \min_{i \in I} \mathbb{E}[|\mathrm{ALG(i)}|] \leq \max_{\mathrm{DALG} i \sim U(I)} \mathbb{E}[|\mathrm{DALG}(i)|] = \mathbb{E}[|\mathrm{RGA}(I^\star)|] \quad (3.1)$$

Since (as proven in the previous section) only instances with perfect matchings need be considered in competitiveness analysis, competitiveness of all algorithms is upper bounded by $\frac{\mathbb{E}[|\mathrm{RGA}(I^\star)|]}{n}$

Now, all that is required to prove the following lemma

Lemma 3.2.2. $\mathbb{E}[|\mathrm{RGA}(I^\star)|] \leq n(1 - \frac{1}{e}) + o(1)$

The rest of this section is the proof of this lemma.

Let $\mathrm{rem}(t)$ be the random variable representing the number of remaining online vertices and $\mathrm{avl}(t)$ be the number of offline vertices available for matching, at the time t. Δf is used conventionally, as the change of $f(t)$ over t.
Δrem is always -1 while Δavl is either -1 or -2. It is only -2 if n_t is being matched and f_t is available (event A_t^1) and it isn't matched to n_t (event A_t^2) since f_t becomes unmatchable.
Note that rem is essentially a clock ticking backwards since it has value $n{-}t$

$\Pr[A_t^1 | \mathrm{avl}, \mathrm{rem}] = \frac{\mathrm{avl}}{\mathrm{rem}+1}$ since from Claim 1 from Lemma 3.2.1 we know that all avl sized subsets are equally likely to be the available subset
$\Pr[A_t^2 | \mathrm{avl}] = 1 - \frac{1}{\mathrm{avl}} = \frac{\mathrm{avl}-1}{\mathrm{avl}}$ since matched vertex is chosen randomly
Since both events are independent $\Pr[A_t^1 \wedge A_t^2] = \frac{\mathrm{avl}-1}{\mathrm{rem}+1}$
$\mathbb{E}[\Delta\mathrm{avl}] = (-2)\frac{\mathrm{avl}-1}{\mathrm{rem}+1} + (-1)(1 - \frac{\mathrm{avl}-1}{\mathrm{rem}+1})$

finally

$$\mathbb{E}[\Delta \text{avl}] == -1 - \frac{\text{avl}(t) - 1}{\text{rem}(t)} \tag{3.2}$$

implying $\frac{\mathbb{E}[\Delta \text{avl}]}{\mathbb{E}[\Delta \text{rem}]} = 1 + \frac{\text{rem}(t)-1}{\text{avl}(t)}$ since Δrem is always -1
Though this is a discrete Markov Chain, we can approximate it by treating it as a continuous function, whose rate of growth is already known. Moreover, from Kurtz's theorem ([Kur70]) this approximation's accuracy increases as n increases

$$\frac{d\text{avl}}{d\text{rem}} = 1 + \frac{\text{avl} - 1}{\text{rem}} \tag{3.3}$$

We also have the initial conditions, avl = rem = n, meaning we can find an approximating equation by solving *Equation* 3.3

$$\begin{aligned}
\frac{d\text{avl}}{d\text{rem}} = 1 + \frac{\text{avl} - 1}{\text{rem}} &\text{ can be rewritten as} \\
\frac{d\text{avl}}{d\text{rem}} - \frac{\text{avl}}{\text{rem}} = \frac{\text{rem} - 1}{\text{rem}} &\text{ multiply both sides by } m(\text{rem}) = e^{\int -1/\text{rem}\, d\text{rem}} \\
\frac{\frac{d\text{avl}}{d\text{rem}}}{\text{rem}} + \frac{d}{d\text{rem}}\left(\frac{1}{\text{rem}}\right)\text{avl} = \frac{\text{rem} - 1}{\text{rem}^2} &\text{ apply product rune to LHS} \\
\frac{d}{d\text{rem}}\left(\frac{\text{avl}}{\text{rem}}\right) = \frac{rem - 1}{\text{rem}^2} &\text{ integrate both sides} \\
\int \frac{d}{d\text{rem}}\left(\frac{\text{avl}}{\text{rem}}\right) = \int \frac{\text{rem} - 1}{\text{rem}^2} & \\
\frac{\text{avl}}{\text{rem}} = \frac{1}{\text{rem}} + \ln \text{rem} + c &\text{ apply initial condition avl} = \text{rem} = n \\
1 = \frac{1}{n} + \ln n + c \implies c = 1 - \frac{1}{n} - \ln n \implies & \\
\text{avl} = 1 + \text{rem}\left(\ln \text{rem} - \ln n + 1 - \frac{1}{n}\right) &
\end{aligned}$$

giving us the final approximation

$$\text{avl} = 1 + \text{rem}\left(\frac{n-1}{n} - \ln \frac{\text{rem}}{n}\right) \tag{3.4}$$

Bearing in mind that the initial Markov chain doesn't perfectly emulate the last step, we substitute rem = 1 instead rem = 0 and from Equation 3.4 we receive that, when only 1 online vertex has yet to arrive, the number of remaining columns is $n\frac{1}{e} + o(1)$. Bearing in mind that the last arriving vertex will affect the performance by at most 1, the expected size of the final matching is $n\left(1 - \frac{1}{e}\right)$

Chapter 4

Random Arrival

As shown before, relatively simple algorithms achieve the optimal competitiveness of $1 - \frac{1}{e} \approx 0.632$. A natural question is to ask how to break this barrier. This is possible with a relatively small model change.

Definition 4.0.1 (Random Arrival model)**.** *The random arrival model is identical to the semi-Online model except for one crucial difference - the order of arrival of vertices is random instead of adversarial. All other details remain identical*

In this section we will prove that both Ranking and RGA perform strictly better in the Random Arrival model than in the Semi-Online model

Theorem 3. *In the random arrival model, RGA(Algorithm 3) has a competitiveness of* $1 - \frac{1}{e}$

Theorem 4. *In the random arrival model, Ranking has a competitiveness of at least* 0.696

4.1 RGA

We will present a proof of Theorem 3 by [GM08] which goes in three phases, first proving properties of RGA, then more 3 complex lammas and finally using those to prove the theorem with the help of linear programming

For ease of argumentation we will describe randomness of the algorithm not as an ability to make a truly random choice, but as seeded randomness, meaning that a deterministic algorithm is given an arbitrary number at initialization which can be used to make random decisions and is unknown to the adversary. The latter description of randomness is identical to the former in practice but it makes discussing runs on inputs easier easier. When runs are compared they can be assumed to have the same random seed.

The proof is in three parts.

4.1.1 basic properties

We will begin by proving several properties of RGA.

Lemma 4.1.1 (Prefix)**.** $\forall \sigma_1, \sigma_2 \in \text{Perm}([n]), t \in [n] | (\forall 0 \leq k \leq t | \sigma_1(k) = \sigma_2(k)) \implies$ *runs of RGA on* σ_1 *and* σ_2 *are identical*

Proof. This lemma is relatively trivial. RGA doesn't have access to σ so, while both permutations are identical the runs will be identical as well. □

Let Ω_p be a set of permutations where all vertices but v_p have fixed positions, $\sigma_k \in \Omega_p$ be the permutation where x_p is on position k, $G_s(t)$ be the set of offline vertices matched in time $1, \ldots, t$ by RGA on σ_s

Lemma 4.1.2 (Monotonicity)**.** $\forall 1 \leq s < m \leq n$*:*

- $\forall 1 \leq t \leq s-1$ $G_m(t) = G_s(t)$
- $\forall s-1 \leq t \leq m-1$ $G_m(t) \subseteq G_s(t+1)$

Another way of phrasing this lemma is to say that, if a vertex is moved up in the permutation then all offline vertices matched previously by time t are expected to be matched by time $t+1$

Proof. The first part is a direct consequence of Lemma 4.1.1. Proof of the second part will be by induction over time t, starting with $s-1$ which is directly implied by the first part.
Assuming the lemma holds at time k, $G_m(k) \subseteq G_s(k+1)$. Notice that $\sigma_m(k+1) = \sigma_s(k+2)$. Denote $v := v_{\sigma_m(k+1)}$, matched to u on RGA run on σ_m. Suppose $u \notin G_s(k+2)$. Then u was available when RGA was matching v on σ_s, but v was instead matched to $u' \notin G_s(k+1)$. By induction hypothesis $u' \notin G_m(k)$ implying u, u' available at $t = k+1$ in run σ_m, RGA should have chosen u' over u, contradiction implying $u \in G_s(k+2)$ implying step condition for $t = k+1$ □

As context for the last lemma, we need to establish the following: what it means to miss a vertex, what is a good match and what is a bad match.

A vertex is considered missed if it remains unmatched by the end of the run

$$\text{miss}_\sigma(s,p) = \begin{cases} 1 & \sigma(s) = p \wedge \ v_p \text{ unmatched at the end of RGA run on } \sigma \\ 0 & \text{else} \end{cases}$$

if v_p was missed then u_p must have been matched to some $v_{p'}$ and $v_{p'}$ arrived before v_p. We define this match (between u_p and $v_{p'}$) as a bad match since in the end it caused v_p to be missed

$$\text{bad}_\sigma(s,q) = \begin{cases} 1 & u_q \text{ is matched to } v_{\sigma(s)} \wedge \sigma^-1(q) > s \\ 0 & \text{else} \end{cases}$$

in contrast, a good match occurs when $v_{p'}$ arrives after v_p. In that situation we have a guarantee that v_p wasn't missed since u_p and v_p are connected

$$\text{good}_\sigma(s,q) = \begin{cases} 1 & u_q \text{ is matched to } v_{\sigma(s)} \wedge \sigma^-1(q) \leq s \\ 0 & \text{else} \end{cases}$$

Lemma 4.1.3 (Partitioning)**.** *$\forall p, \Omega_p$, during a run of RGA on σ_n. If u_p is matched to v_t then*

$$\forall x \in [1,t] : \sum_{1 \leq s \leq t+1} \text{good}_{\sigma_x}(s,p) = 1 \forall x \in [t+1,n] : \quad \text{bad}_{\sigma_x}(t,p) = 1 \quad (4.1)$$

Proof. 1. from Lemma 4.1.2 we know that $\forall x \in [1,t]$, in a run on σ_x is matched to v_k for some $k \in [x, t+1]$ u_p is matched to v_t implying inequality 1 2. from Lemma 4.1.1 we know that $\forall x \in [t+1,n]$, in a run on σ_x u_p is matched to some v_k for $k \in [x, t+1]$ implying inequality 2 □

4.1.2 bad/good/miss properties

The second part is to prove three helper lemmas about miss, bad, good with the basic properties

Lemma 4.1.4. $\forall \sigma \in \text{Perm}([n]), t \in [n]$
$\sum_{1 \leq p \leq n} \text{miss}_\sigma(t,p) + \sum_{1 \leq p \leq n} \text{bad}_\sigma(t,p) + \sum_{1 \leq p \leq n} \text{good}_\sigma(t,p) = 1$

Proof. this lemma is rather trivial since a vertex is either missed or not, and if it is then either $p < p', p > p'$ or $p = p'$ □

Lemma 4.1.5. $\forall t \in [n]$
$\text{miss}_{\sigma_t}(t,p) \leq \sum_{\sigma \in \Omega_p} \sum_{s<t} \frac{\text{bad}_\sigma(s,p)}{n-s}$

Proof. If miss is 0 then the inequality is trivially true since bad and $n-s$ are non-negative.
Assume miss = 1 from definition, in run on σ_t v_p that is on position t remains unmatched, which is only possible if u_p is matched to $v_{p'}$ on position t', $t' < t$

using Lemma 4.1.3 we have

$$\begin{aligned}
&\sum_{\sigma\in\Omega_p}\sum_{s<t}\frac{\mathrm{bad}_\sigma(s,p)}{n-s}\\
\geq&\sum_{\sigma\in\Omega_p}\frac{\mathrm{bad}_\sigma(t',p)}{n-t'} && t'<t\\
\geq&\sum_{t'+1\leq s\leq n}\frac{\mathrm{bad}_{\sigma_s}(t',p)}{n-t'}\\
=&1 && \text{Lemma 4.1.3.2}\\
=&\mathrm{miss}_{\sigma_t}(t,p)
\end{aligned}$$

concluding the proof □

Lemma 4.1.6. $\forall t\in[n-1]$: $\sum_{\sigma\in\Omega_p}\sum_{s\leq t}\mathrm{bad}_\sigma(s,p)\frac{s}{n-s}\leq\sum_{\sigma\in\Omega_p}\sum_{s\leq t+1}\mathrm{good}_\sigma(s,p)$

Proof. **Case 1** u_p remains unmatched on a run on σ_n
Let $\sigma_x\in\Omega_p$, if u_p is matched in a run on σ_x then by Lemma 4.1.1 it has to be matched to $v_{x'}, x'\leq x$ implying $\forall t|\mathrm{bad}_{\sigma_x}(t,p)=0$ and the lemma proof becomes trivial by non-negativity

Case 2 u_p is matched in a run on σ_n to $v_{t'}$. We divide this case into two following cases

Case 2.1 $t<t'$
From Lemma 4.1.3 and Lemma 4.1.4 we know which variables are 1, and that for every $\sigma\in\Omega_p, s<t', \mathrm{bad}_\sigma(s,p)=0$, implying LHS is 0 and the Lemma is again trivial

Case 2.2 finally, if $t\geq t'$

$$\begin{aligned}
&\sum_{\sigma\in\Omega_p}\sum_{s\leq t}\mathrm{bad}_\sigma(s,p)\frac{s}{n-s}\\
&=\sum_{\sigma\in\Omega_p}\mathrm{bad}_\sigma(t',p)\frac{t'}{n-t'}\\
&=t'\\
&=\sum_{\sigma\Omega_p}\sum_{s\leq t'+1}\mathrm{good}_\sigma(s,p)\\
&=\sum_{\sigma\Omega_p}\sum_{s\leq t+1}\mathrm{good}_\sigma(s,p)
\end{aligned}$$

□

4.1.3 final proof

From here we have the tools to prove the Theorem 3

Notice that the total size of the matching is $\sum_s E\left[\sum_p \text{good}_\sigma(s,p)\right] + \mathbb{E}\left[\sum_p \text{bad}_\sigma(s,p)\right]$.

Informally, the three lemmas suggest that if there are many misses, then there must be many good and bad matches, bounding how small the expected matching can be.

We will start by introducing generalized miss/bad/good variables

$$\text{miss}(s) = \mathop{\mathbb{E}}_{\sigma\sim\text{Perm}([n])}\left[\sum_p \text{miss}_\sigma(s,p)\right]$$

$$\text{bad}(s) = \mathop{\mathbb{E}}_{\sigma\sim\text{Perm}([n])}\left[\sum_p \text{bad}_\sigma(s,p)\right]$$

$$\text{good}(s) = \mathop{\mathbb{E}}_{\sigma\sim\text{Perm}([n])}\left[\sum_p \text{good}_\sigma(s,p)\right]$$

By using Lemma 4.1.5 and summing over all partitions Ω_p and all p we receive thee following

$$\forall t : \sum_p \sum_{\sigma\in\Omega} \text{miss}_\sigma(t,p) \le \sum_p \sum_{\sigma\in\Omega} \sum_{s<t} \frac{\text{bad}_\sigma(s,p)}{n-s}$$

change summation order, divide by $|\Omega|$

$$\forall t : \text{miss}(t) \le \sum_{s<t} \frac{\text{bad}(s)}{n-s}$$

by doing the same on Lemma 4.1.6 instead we get

$$\forall t : \sum_{s\le t} \text{bad}(s)\frac{s}{n-s} \le \sum_{s\le t+1} \text{good}(s)$$

with these inequalities we can rewrite the lower bound on expected matching size as a solution of a linear program

$$
\begin{aligned}
\text{minimize} \quad & \sum_t [\text{bad}(t) + \text{good}(t)] \\
\text{subject to} \quad & \forall t < n \sum_{s \le t+1} \text{good}(s) - \sum_{s \le t+1} \text{bad}(s) \frac{s}{n-s} && \ge 0 \\
& \forall t \le n \text{good}(t) + \text{bad}(t) + \sum_{s<t} \frac{\text{bad}(s)}{n-s} && \ge 1 \\
& \text{good}(t), \text{bad}(t), \text{miss}(t) && \ge 0
\end{aligned}
$$

which can in turn be rewritten with $m_t = \text{good}(t) + \text{bad}(t), v_t = \frac{\text{bad}(t)}{n-t}$ as the following

$$
\begin{aligned}
\text{minimize} \quad & \sum_t m_t \\
\text{subject to } \forall t < n \quad & \sum_{s \le t+1} m_s - n \sum_{s \le t} v_s \ge 0 \\
\forall t \le n \quad & m_t + \sum_{s<t} v_s \ge 1 \\
& m_t, v_t \ge 0
\end{aligned}
$$

Dropping the m_{t+1} from $\sum_{s \le t+1} m_s$ narrows down the solution space, so the optimum of a program with $\sum_{s \le t} m_s$ will give a lower bound on the expected competitiveness. The following is that program, alongside it's dual. In fact, [KVV90] prove that this modification doesn't change the optimum in any significant way, but this isn't necessary for the purposes of this proof.

$$
\begin{aligned}
\text{minimize} \quad & \sum_t m_t \\
\text{subject to } \forall t < n \quad & \sum_{s \le t} m_s - n \sum_{s \le t} v_s \ge 0 \\
\forall t \le n \quad & m_t + \sum_{s<t} v_s \ge 1 \\
& m_t, v_t \ge 0
\end{aligned}
\qquad
\begin{aligned}
\text{maximize} \quad & \sum_i \alpha_i \\
\text{subject to} \quad & \forall t < n \sum_{s>t} \alpha_s - n \sum_{s \ge t} \beta_s && \le 0 \\
& \forall t \le n \ \alpha_t + \sum_{s \ge t} \beta_s && \le 1 \\
& \alpha_t, \beta_t && \ge 0
\end{aligned}
$$

Notice that both primal and dual constraints are satisfied when all inequalities are tight, which in turn implies the optimality of that solution by weak duality.

$$
m_t = \alpha_{n+1-t} = (1 - \frac{1}{n})^{i-1} \qquad v_t = \beta_{n+1-t} = \frac{1}{n}(1 - \frac{1}{n})^{i-1}
$$

with the objective being $\sum m_t \approx n(1-\frac{1}{e})$, concluding the proof.

This result should not be surprising, as in [KVV90] Greedy with random arrivals is used to lower bound Ranking via duality principles. However, this proof style of proving properties, creating a linear program based on those and then analyzing the program is still useful since duality principles can't be used to prove the competitiveness of Ranking in the Random Arrival model and this more direct method has to be used instead.

4.2 Ranking

In this section we will prove Theorem 4 following an approach by [MY11]

The approach is to

1. prove two inequalities, which are properties of the Ranking algorithm in the Random Arrival model
2. use these to define a family of exponential size linear programs expLP(n,k)
3. relax it to a family of polynomial size linear programs PolyLP(n)
4. derive a family of Strongly Factor-Revealing linear program PolyLP$'(n)$
5. use computing machines to solve some linear programs from PolyLP$'(n)$ to obtain a lower bound

Do note that the last step is computational and does not prove an exact competitiveness ratio. Indeed, this approach merely shows that the ratio is at least 0.696, and other analysis show it can be no larger than 0.727. The exact competitiveness ratio is currently unknown.

4.2.1 Ranking properties

For the purposes of this section we fix a graph $G = (V, U, E)$
$x(\sigma, \pi, u, v) = 1$ iff $(u, v) \in \text{Ranking}(G, \sigma, \pi)$
$m^\star(u, v) = 1$ iff $M^\star(u) = v$.
We will abuse notation slightly and assume $U = [n], V = [m]$ for some $n, m \in \mathbb{N}$, in this way u is both a vertex and it's index in an arbitrary ordering. We remind that this means that $\sigma(u)$ is the vertex at rank u

Lemma 4.2.1 (Dominance)**.** $\forall \sigma \in \text{Perm}(U), \pi \in \text{Perm}(V), u \in U, v \in V$

$$\sum_{1 \le u' \le u} x(\sigma, \pi, u', v) + \sum_{1 \le v' \le v-1} x(\sigma, \pi, u, v') \ge m^\star(\sigma(u), \pi(v)) \tag{4.2}$$

Proof. This is an observation made before in the proof of Lemma 3.1.2 $(\sigma(u), \pi(v)) \in M^\star$. When v arrives at time $\pi(v) = t$, if it is matched then it must be matched

either to u or to u' with a lower rank. The equation follows directly from the observation □

Let $\text{geq}(u_i, u_j) = 1$ iff $u_i \geq u_j$

Lemma 4.2.2 (Monotonicity)**.** *$\forall \sigma \in \text{Perm}(U), \pi \in \text{Perm}(V), u_{\text{new}}, u_{\text{old}}, u \in U, v \in V$ such that $l_{\text{new}} < l_{\text{old}}, l < l_{\text{old}}$, Let σ' be σ where u_{new} and u_{old} swap places.*

$$\sum_{1 \leq u' \leq u + \text{geq}(u, u_{\text{new}})} x(\sigma', \pi, u', v) \geq \sum_{1 \leq u' \leq u} x(\sigma, \pi, u', v) \tag{4.3}$$

Proof. Again, this is a claim that is similar to the one in Lemma 3.1.3. The difference between $\text{Ranking}(G, \sigma, \pi)$ and $\text{Ranking}(G, \sigma', \pi)$ is a single alternating path starting from $\sigma(u_{\text{old}})$, implying that if v was matched to some offline vertex with rank at most u in $\text{Ranking}(G, \sigma, \pi)$ then in $\text{Ranking}(G, \sigma', \pi)$ it is matched an offline vertex with rank at most $u + \text{geq}(u, u_{\text{new}})$, which directly implies the inequality □

We can also express a symmetrical equation for π'

$$\sum_{1 \leq v' \leq v + \text{geq}(v, v_{\text{new}})} x(\sigma, \pi', u', v) \geq \sum_{1 \leq v' \leq v} x(\sigma, \pi, u, v') \tag{4.4}$$

the proof is symmetrical, so we will omit it

4.2.2 formulating $\text{expLP}(n, k)$

for the purposes of creating $\text{expLP}(n, k)$ for a graph G with n vertices on both sides and k edges, for every $\sigma, \pi, u, v \in \text{Perm}(U) \times \text{Perm}(V) \times U \times V$, $x(\sigma, \pi, u, v)$ is a separate variable. We create a linear program that enforces the properties of x as follows

$$\begin{aligned}
\text{minimize} & \frac{1}{(n!)^2 k} \sum_{\sigma, \pi, u, v} x(\sigma, \pi, u, v) \\
\text{subject to} & \textit{Equation } 4.2 \\
& \textit{Equation } 4.3 \\
& \textit{Equation } 4.4 \\
& x(\sigma, \pi, u, v) \geq 0 \\
& x(\sigma, \pi, u, v) \leq 1
\end{aligned}$$

Lemma 4.2.3. *The competitiveness ratio of Ranking in the Random Arrival model is lower bounded by $\inf_{n \in N} \inf_{l} \text{expLP}(n, k)$ for any infinite subset of natural numbers N*

Proof. Let $n \in \mathbb{N}, n' \in \{x \in N | x > n\}$.
Suppose we want to run Ranking on a $n \times n$ graph G. By adding isolated vertices we can change it into a $n' \times n'$ graph with the identical run to that on G when ignoring the isolated vertices
As shown in the lammas in the previous section, a valuation of x is a feasible solution to expLP(n, k). The value of the objective function is the average over all σ and π divided by n, making it the performance of Ranking on that graph. Note that the linear program is a relaxation, and so provides a lower bound on the competitiveness ration □

4.2.3 relaxation to PolyLP(n)

This method relies in the final step on using LP solvers to obtain a bound rather then proving it analytically. For this reason the exponential size of expLP(n, k) relative to n is unfortunate as it will make it practically impossible to complete this computation in a reasonable time. For this reason we will create a family of polynomial size linear programs which in turn provide a good lower bound on the optimal values of the objective function of expLP(n, k)

We assume WLOG that the optimal matching $M^\star = \{(i, i) | i \in [n]\}$

Let $x_1(u, v, p)$ be the probability over random $\sigma, \pi \in \text{Perm}(U) \times \text{Perm}(V)$ that offline agent of rank u is matched to an online agent at time v by Ranking and the offline agent at rank p is matched to the online agent arriving at time v in $M^\star$

Let $x_2(u, v, q)$ be the probability over random $\sigma, \pi \in \text{Perm}(U) \times \text{Perm}(V)$ that offline agent of rank u is matched to an online agent at time v by Ranking and that the online agent arriving at time q will be matched to the offline agent at rank l in $M^\star$

using $\text{eq}(x, y) = 1$ iff $x = y \wedge x, y < k$ where k is the amount of edges, we can define these variables

$$x_1(u, v, p) = \mathop{\mathbb{E}}_{\sigma,\pi}[\text{eq}(\sigma(p), \pi(v))x(\sigma, \pi, u, v)] x_2(u, v, p) = \mathop{\mathbb{E}}_{\sigma,\pi}[\text{eq}(\sigma(u), \pi(q))x(\sigma, \pi, u, v)]$$

We introduce partial sum variables

$$\forall i \in \{1, 2\} \qquad y_i(u, v, p) = \begin{cases} 0 & \text{if } u = 0 \vee v = 0 \\ \sum\limits_{1 \le u' \le u} x_i(u', r, p) & \text{else} \end{cases}$$

Now we derive the new inequalities for PolyLP(n) multiplying both sides of Equation 4.2 by $\text{eq}(\sigma(u) m \pi(r))$ gives

$$\sum_{1\le u'\le u} \mathrm{eq}(\sigma(u),\pi(r))x(\sigma,\pi,u',v)+\sum_{1\le v'\le v} \mathrm{eq}(\sigma(u),\pi(r))x(\sigma,\pi,u,v') \ge \mathrm{eq}(\sigma(u),\pi(v))$$

and taking expectation over random σ, π results in

$$y_i(u,v,u) + y_2(v-1,u,v) \ge \frac{k}{n^2} \tag{4.5}$$

We multiply Equation 4.3 similarly, for a fixed p let $u_{\text{old}} = n$ and again take expectation over random σ, π. RHS is $y_1(u,v,p)$.

Notice that σ' is uniformly random, $\sigma(p) = \pi(v)$ iff $\sigma''(p') = \pi(r)$ where σ'' which σ with u_{new}, n swapped and

$$p' = \begin{cases} p & p < u_{\text{new}} \\ p+1 & u_{\text{new}} \le p < n \\ u_{\text{new}} & p = n \end{cases}$$

from which we get the following equation

$$y_1(u + \mathrm{geq}(u, u_{\text{new}}), v, p') \ge y_1(l,r,p) \tag{4.6}$$

and symmetrically from *Equation* 4.4

$$y_2(v + \mathrm{geq}(v, v_{\text{new}}), u, q') \ge y_1(l,r,q) \tag{4.7}$$

with q' being defined symmetrically to p'. Finally we add

$$\sum_p x_1(u,v,p) = \sum_q x_2(v,u,q)$$

as both sides are equal to $\frac{n}{k}\mathop{\mathbb{E}}_{\sigma,\pi}[x(\sigma,\pi,u,v)]$. Together, these inequalities also imply non-negativity and with y_i replaced with their definitions on x_i these inequalities form the LP constraints.

The objective function is defined as $\frac{1}{2k}\left(\sum_{u,v,p} x_1(u,v,p) + \sum_{u,v,q} x_2(v,u,q)\right)$, equivalent to the previous objective.

Before continuing to the last step we will implement the following simplifications to PolyLP(n).

- $x_1 = x_2$ since for a feasible x_1, x_2 $x'_1 = x'_2 = \frac{x_1+x_2}{2}$ is also feasible and this reduces the amount of variables by 2
- WLOG $k = n$ since the fact that a solution for k can be rescaled to k' makes k inconsequential
- and several other which are more technical

which finally results in

$$\begin{aligned}
&\text{Minimize} \frac{1}{n} \sum_{u,v,p \in [n]} x(u,v,p) \\
&\text{subject to} y(u,v,u) + y(v-1u,v) \geq \frac{1}{n} \\
&\qquad y(u+1,v,p+1) \geq y(u,v,p) && \forall p \leq l < n \\
&\qquad y(u,v,p) = y(u,v,u+1) && \forall u < p \\
&\qquad y(u+1,r,p) \geq y(u,v,u+1) && \forall p \leq u < n \\
&\qquad \sum_p x(u,v,p) = \sum_p x(v,u,p)
\end{aligned}$$

4.2.4 deriving PolyLP$'(n)$

For a maximization algorithm a family of Linear Programs is strongly factor-revealing iff the solution of any linear program in it is a lower bound on the approximation ratio of the algorithm. It is stronger than the more common factor revealing family for which the infimum of the optimum solutions is the lower bound.

PolyLP(n) is factor-revealing, in this section we will transform it into a strongly factor-revealing family PolyLP$'(n)$. To do this, we will replace the first constraint with

$$y(u,v,u) + y(v,u,p) \geq \frac{1}{n} \tag{4.8}$$

Lemma 4.2.4. *the family of linear programs* PolyLP$'(n)$ *is strongly factor-revealing for Ranking with random arrivals*

Proof. Take any integer $m > 0$. For any instance of size $n = km, k \in \mathbb{Z}$. We will prove that the competitiveness ratio of Ranking is lower-bounded by PolyLP$'(m)$. We know that $\inf_d$ PolyLP(md) is greater or equal to the desired competitiveness ratio. We will take an optimal solution (x, y) of PolyLP(md) and project it to a feasible solution of PolyLP$'(m)$ as follows

Definition 4.2.1. *For any* $i, j, k \in [m]$ *define* $f(i) = \{d(i-1)+1, \ldots, d*i\}, f(i,j) = f(i) \times f(j), f(i,j,k) = f(i) \times f(j) \times f(k)$ *(*$\times$ *denotes Cartesian product)*
For any $u', v', p' \in [m]$ *define* $x'(u', v', p') = \frac{1}{d} \sum_{(u,v,p) \in f(u',v',p')} x(u,v,p)$,$y'(u', v', p')$ $= \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, v, p)$

We will show what x', y' for PolyLP$'(m)$ has the same objective value as x, y for PolyLP(m)

$$\begin{aligned}
&\frac{1}{m} \sum_{u',v',p' \in [m]} x'(u', v', p') \\
=&\frac{1}{m} \sum_{u',v',p' \in [m]} \frac{1}{d} \sum_{(u,v,p) \in f(u',v',p')} x(u,v,p) \\
=&\frac{1}{n} \sum_{u,v,p \in [n]} x(u,v,p) \\
=&\text{PolyLP}(n)
\end{aligned}$$

Next, we show that x', y' is feasible in PolyLP(n)
The fifth constraint of PolyLP$'(n)$ can be obtained by summing the fifth constraint of PolyLP(n) over $(u, v) \in f(u', v')$
The sixth constraint of PolyLP$'(n)$ can be obtained by summing the sixth constraint of PolyLP(n) over $(v, p) \in f(v', p'), u = u'd$
The second constraint of PolyLP$'(n)$ can be obtained as follows: for $p' \leq u' < m$

$$\begin{aligned}
y'(u'+1, v', p'+1) &= \frac{1}{d} \sum_{(r,p) \in f(v',p')} y(u'd+d, v, p+d) \\
&\geq \frac{1}{d} \sum_{(r,p) \in f(v',p')} y(u'd, v, p) \\
&= y'(u', v', p')
\end{aligned}$$

where the inequality is the result of chaining multiple $y(u'd+j+1, r, p+j+1) \geq y(u'd+j, v, p+j)$ inequalities
The third constraint of PolyLP$'(n)$ can be obtained as follows: for $u' < p' \leq n$, $r' \leq n$

$$\begin{aligned}
y'(u', v', p') &= \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, v, p) \\
&= \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, r, u'd+1) \\
&= \frac{1}{d} \sum_{(v,p) \in f(v',u'+1)} y(u'd, v, p) \\
&= y'(u', v', u'+1)
\end{aligned}$$

The fourth constraint of PolyLP$'(n)$ can be obtained as follows: for $p' \leq u' < n, v' \leq n$

$$\begin{aligned}
y'(u'+1, v', p') &= \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd + d, v, p) \\
&\geq \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd + 1, v, p) \\
&\geq \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, v, u'd + 1) \\
&= \geq \frac{1}{d} \sum_{(v,p) \in f(v',p')} y(u'd, v, p) \\
&= y'(u', v', u' + 1)
\end{aligned}$$

where the first inequality is the consequence of the definition of y and x's non-negativity and the second inequality follows from the fourth constraint of PolyLP(n)

The first constraint of PolyLP$'(n)$, which is Equation 4.8, can be obtained as follows

$$\begin{aligned}
y'(v', u', p') &= \frac{1}{d} \sum_{(u,p) \in f(u',p')} y(v'd, u, p) \\
&\geq \frac{1}{d} \sum_{(u,p) \in f(u',p')} y(v'd - 1, u, v'd) \\
&\geq \frac{1}{d} \sum_{(u,p) \in f(u',p')} y(v - 1, u, v'd) \\
&= \frac{1}{d} \sum_{(u,p) \in f(u',p')} y(v - 1, u, v)
\end{aligned}$$

where the first inequality is the consequence of the fact that $p \geq v'd \implies y(v'd, u, p) \geq y(v'd-1, u, p) = y(v, d-1, u, v'd)$ and $p < v'd \implies y(v'd, u, p) \geq y(v'd-1, u, v'd)$. The second inequality is the consequence of the third constraint of PolyLP(n)

Concluding

$$\begin{aligned}
& y'(u', v', u') + y'(v', u', p') \\
\geq & \frac{1}{d} \sum_{(v,u) \in f(v',u')} y(u'd, v, u) + \frac{1}{d} \sum_{(u,v) \in f(u',v')} y(v - 1, u, v) \\
\geq & \frac{1}{d} \sum_{(v,u) \in f(v',u')} y(u, v, u) + \frac{1}{d} \sum_{(u,v) \in f(u',v')} y(v - 1, u, v) \\
\geq & \frac{1}{d} d^2 \frac{1}{n} = \frac{1}{m}
\end{aligned}$$

where the first inequality is a consequence of non-negativity of x and the definition of y and the second inequality is a result of the first constraint in PolyLP(n)

With this we conclude - for all PolyLP(md), their optimal solutions are mapped to feasible solutions of PolyLP$'(m)$, making the optimum of PolyLP$'(m)$ a lower bound on $\inf_d$ PolyLP(md), a lower bound on competitiveness $\square$

With Lemma 4.2.4 proven, the theoretical framework of this proof is complete and we cite the computational result. In [MY11], appendix B state that the optimum solution for PolyLP$'(50)$ is 0.696150. We also note that these results were obtained using CPLEX software.

Chapter 5

Stochastic Arrivals

Definition 5.0.1 (iid Model)**.** *In the iid model the algorithm always knows a set of offline vertices U, but there is no determined set of online vertices V. Instead, there is a set V' of vertex 'archetypes' consisting of a neighborhood $N \subseteq U$ and a probability p. When a new vertex v arrives, for each $(N, p) \in V'$ v has of p to have neighborhood N. The archetype set V' is known to the algorithm. The amount of vertices that will arrive in total, n, is also known.*

We note two things here:

1. obviously, $\sum_{(N,p)\in V'} p = 1$
2. This model is still a game, but the adversary has been stripped of almost all agency and now cannot even control the final graph

We will present the proof of two theorems in this chapter:

Theorem 5. *No algorithm has a competitiveness of $1 - \varepsilon$ for an arbitrary ε in the iid model*

meaning that there is no perfect algorithm, and

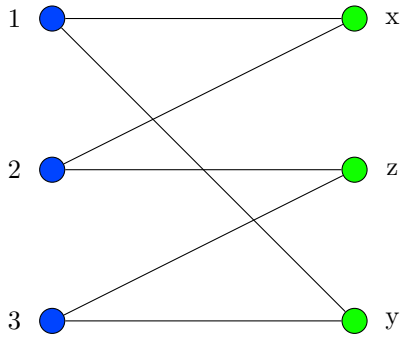
Theorem 6. *The stochastic arrival model is strictly easier than the Semi-Online model since there is an algorithm, TSM, with competitiveness ≈ 0.67029*

In actuality, there is an algorithm with a competitiveness of 0.702, better than the proven competitiveness of Ranking in the random arrival model. However, this algorithm is very complicated so we will instead showcase a simpler one to show that the $1 - \frac{1}{e}$ barrier can be beaten

5.1 Competitiveness Limit

In this section we will follow a proof from [Fel+09]. Consider a 6-Cycle defined as follows:

Definition 5.1.1. $U = \{1, 2, 3\}, V' = \{x = (\{1, 2\}, \frac{1}{3}), y = (\{1, 3\}, \frac{1}{3}), z = (\{2, 3\}, \frac{1}{3})\}$



If x arrives first, if it is matched to 1 and then two y vertices y_1, y_2 arrive, the second will be unmatchable. In this scenario there is a perfect matching $M^\star = \{(2, x), (1, y_1), (3, y_2)\}$

This argument is symmetric, for any first arrival v matched to u there is an archetype v' that forces this scenario.
The probability of this scenario is therefore the probability of (v', v') arriving after v, which is $\frac{1}{9}$. Assuming that in all other scenarios the algorithm find the perfect matching, competitiveness is ar most $\frac{1}{9}\frac{2}{3} + \frac{8}{9}1 = \frac{26}{27}$.

Therefore, the theorem is proven and the competitiveness of any algorithm in the iid model is bounded by $\approx 0.962 < 1$

Remark 5.1.1. *We can extend this result asymptotically (albeit not the exact upper bound) by duplicating the 6-Cycle k times*

5.2 TSM Algorithm

We present the algorithm from [Fel+09].
For the purposes of simplicity we assume that all $p_{v_i} \approx \frac{1}{n}$. Any case where this is not the case can be reduced by duplicating an archetype and dividing both probabilities as appropriate. We may assume that $p_{v_i} n \in \mathbb{Z}$ for all $i \in [n]$ for simplicity This algorithm works on the basis of creating a realization graph, where there is one vertex from each archetype and then building two different solutions that will be used to guide the algorithm as vertices arrive

We note that any $v \in V$ is adjacent to either: no colored edges, a blue edge or a blue and red edge. To analyze the performance of this algorithm we will first

Algorithm 4 TSM

on Initialization:
construct realization graph G, build boosted flow graph G_f as follows: create source s connected to all offline vertices, sink t connected to all online vertices, set capacity of edges as follows

1. edges (s, v) from source have capacity 2
2. edges (t, u) from sink have capacity 2
3. regular edges (u, v) have capacity 1

Find maximum flow of this graph. Let E_f be the regular edges with non-zero flow. Assign colors, blue or red, to E_f as follows

- cycles are colored alternating
- paths of odd length are colored alternating, starting with blue
- paths of even length starting (and ending) in U are colored alternating
- paths of even length starting in V have the first two edges colored blue, then alternating

on Arrival of v of archetype V_i':
try matching along blue or red edges if this is the first/second time a vertex from V_i' has arrived. Otherwise ignore this vertex

bound the performance of the algorithm, then the size of the maximal matching. In the analysis we will be using two facts

Fact 5.2.1. *If n balls are thrown in n bins i.i.d with random uniform probability over bins, B is subset of bins, S random variable is the number of bins with at least one ball, then, for any $\varepsilon > 0$, with probability no smaller than $1 - 2e^{-\varepsilon n/2}$ $|B|(1 - \frac{1}{e}) - \varepsilon \leq S \leq |B|(1 - \frac{1}{e} + \frac{1}{en}) + \varepsilon n$*

Fact 5.2.2. *If n balls are thrown in n bins i.i.d with random uniform probability over bins, $B_i \ldots, B_l$ are ordered sequences of bins of size c such that no bin is in more than d sequences, for arbitrary subset $R \subseteq \{1, \ldots, c\}$, we say bin sequence $B_a = (b_1, \ldots, b_c)$ is satisfied if*

1. *at least one bin with index not in R has at least one ball, or*
2. *at least one bin with index in R has at least two balls*

Let S be random variable that equals to the number of satisfied bin sequences. With probability no less than $1 - 2e^{-\varepsilon^2 n/2}$ $S \geq l(1 - \frac{2^{|R|}}{e^c}) - \varepsilon dn - \frac{2^{|R|c^2}}{e^c} \frac{l}{n - c^2}$

Algorithm performance bound Let U_{BR} be the offline vertices incident to a blue and red edge, U_{BB} those incident to two blue edges and symmetrically for U_B, U_R

$$|E_f| = 2|U_{BR}| + 2|U_{BB}| + |U_B| + |U_R| \tag{5.1}$$

Consider $u \in U_B$ with blue edge to archetype (u, v'). u is matched iff v from V' is ever drawn, and since the expected amount of arrivals from any archetype os

1 this is also the probability that a particular bin is non-empty in a balls-in-bins problem with n balls and bins. Applying Fact 5.2.1 we conclude that, with a high probability that the number of offline vertices from U_B that are matched is at least $|U_B|(1-\frac{1}{e})\varepsilon n$

We conduct a similar reasoning for U_{BR}. with blue edge to archetype v_1' and red towards archetype v_2': if 1 vertex from v_2' of 2 from v_1' have arrived then u is matched. We apply Fact 5.2.2 with bin sequences being the neighborhood sets of U_{BR} along, in order, blue and red edges $c=2$, $d=2$ since each online vertex is an endpoint of at most 2 colored edges and R being the second (red) bin of the sequence. We conclude that, with high probability, the number of of offline vertices from U_{BR} that are matched is at least $|U_{BR}|(1-\frac{2}{e^2})-\varepsilon n$.

Symmetric arguments give the high probability bounds for U_R: $|U_R|(1-\frac{2}{e})\varepsilon n$ and U_{BB} : $|U_{BB}|(1-\frac{1}{e^2})-\varepsilon n$ implying the bound

$$\text{ALG} \geq (1-\frac{1}{e^2})|U_{BB}|+(1-\frac{2}{e^2})|U_{BR}|+(1-\frac{3}{2e})(|U_b|+|U_R|)-4\varepsilon n \quad (5.2)$$

given that $|U_B| \geq |U_R|$ as creation of blue edges is prioritized

Optimal solution bound We define (S,T), a min $s-t$ cut of G_f. We start with a canonical 'reachability' min $s-t$ cut of G_f where S is the set of nodes reachable in residual graph G_f after finding E_f. We then modify this cut by, for all $v \in V \cap T$, if v is connected to more than one offline vertex in $U \cap S$ we move it over to S. We note that the value of the cut remains constant
We define $U_S := U \cap S$ and U_T, V_S, V_T symmetrically, E_δ as the set of edges between U_S, V_T. We note that

- $E_\delta \subseteq E_f$ since if $E_\delta \ni (u,v)$ has no flow then v would be reachable from s, and therefore not in T
- $v \in V_T$ implies v has no more than one edge in E_δ as a direct consequence of the modifications
- $u \in U_S$ implies u has no more than one incident edge in E_δ as if it had two edges then, since u is reachable from S it must have residual capacity from s directly or from I_S, both are impossible since (s,u) is saturated and both flow edges connect u to I_T

Let U_δ, V_δ be vertices from U, V that are endpoints of some $e \in E_\delta$. From observations we conclude that E_δ is a matching since the min-cut of G_f is composed of E_δ, edges from s to U_T and edges from t to V_S which, by max-flow-min-cut, results in

$$|E_f| = 2(|U_T|+|V_S|)+|E_\delta| \quad (5.3)$$

We will bound the size of the maximal matching in the realization graph $G = (U, V', E')$ by using (S, T), the min-cut of G_f to find a cut. Let G'_f be a directed version of G_f but with all edges having a capacity of 1 any $s - t$ cut in this graph is an upper bound on OPT. Let $D(V'_i)$ be a set of offline vertices of archetype V'_i A cut (S', T') is constructed as follows: $\overline{V_S} = \bigcup_{V' \in V_S} D(i), \overline{V_T} = \bigcup_{V' \in V_T} D(i)$. We remind here that the right side of G are archetypes of online vertices. For Offline vertices we modify the partitioning (U_S, U_T) as follows: for $U'_\delta \subseteq U_\delta$, set of offline vertices u that are incident to some $v \in D(v)$ we set $S' = V_S \cup (U_S \backslash U'_\delta)$ and $T' = V'_T \cup U_T \cup U'_\delta$

To measure the size of (S', T') in G_f we 'pay' 1 for each $u \in V'_S, v \in U_T, v \in U'_\delta$ and note that all edges from $U \cap S'$ to V'_T have been removed resulting in OPT $\leq |V'_S| + |U_T| + |U'_\delta|$
Using Chernoff bound, with probability $1 - e^{-\Omega(n)}$ $|V'_S| \leq |V_S| + \varepsilon n$ occurs, giving a good probability bound on $|V'_S|$
Consider $u \in U_\delta$ and $v \in V_\delta$ matched to it in E_δ. u appears in U'_δ iff v arrives during the runtime, producing a balls-in-bins problem and enabling the use of Fact 5.2.1 to conclude that, with a high probability, $U'_\delta \leq (1 - \frac{1}{e})|E_\delta| + \varepsilon n + O(1)$, giving a good probability bound on U'_δ

By using the two E_f size inequalities we conclude the following

$$\text{OPT} \leq \frac{1}{2}|E_f| + (\frac{1}{2} - \frac{1}{e})|E_\delta| + \varepsilon n = |U_{BR}| + |U_{BB}| + \frac{1}{2}(|U_B| + |U_R|) + (\frac{1}{2} - \frac{1}{e})|E_\delta| + \varepsilon n \tag{5.4}$$

Bounding $|E_\delta|$ For this inequality to be useful we must prove a correlation between the sizes of E_δ and the various U_x, by proving the following

Lemma 5.2.1. $|E_\delta| \leq \frac{2}{3}|U_{BR}| + \frac{4}{3}|U_{BB}| + |U_B| + \frac{1}{3}|U_R|$

Proof. It is sufficient to prove that this inequality holds for all connected components of the graph induced by E_f. We assume that this graph is a single such component, which is a cycle or a path
Let $(u_1, v_1), (u_2, v_2) \in E_\delta \subseteq E_f$ be arbitrary edges. Edges in E_δ are independent, implying that they won't occur consecutively in this path/cycle.
We will prove that there must be at least two edges between them, by contradiction. WLOG (u_2, v_1) is in the component, but $u_2 \in U_S, i_1 \in V_T$, implying that $(u_w, v_1) \in E\delta$, contradicting edge independence

Case 1 The component is a cycle, let k be it's length. Using the reasoning above we conclude that there are no more than $\frac{k}{3}$ edges in E_δ in that cycle. Offline vertices are all in U_{BR} and are exactly half of the vertices in the cycle, therefore $|E_\delta| \leq \frac{2}{3}|A_{BR}|$ and the lemma is true

Case 2 The component is a path, let k be it's length. Symmetrically to above, $E_\delta \leq \lceil \frac{k}{3} \rceil$. We further divide this scenario into cases

Case 2.1 k is odd. Due to edge coloring rule there is 1 offline vertex in U_B and $\frac{k-1}{2}$ in U_{BR} implying $|E_\delta| \leq \lceil \frac{k}{3} \rceil = \left\lceil \frac{2|U_{BR}|}{3} \right\rceil \leq \frac{2}{3}|U_{BR}| + 1 = \frac{2}{3}|U_{BR}| + |U_B|$

Case 2.2 k is even, path starts with an offline vertex. $|U_B| = |U_R| = 1$, $|U_{BR}| = \frac{k}{2} - 1$, therefore $|E_\delta| \leq \lceil \frac{k}{3} \rceil = \left\lceil \frac{2|U_{BR}|}{3} + \frac{2}{3} \right\rceil$ which can in turn be bounded by analyzing $|U_{BR}|$ modulo 3. $|U_{BR}| \equiv_3 0$ implies $|E_\delta| \leq \frac{2}{3}|U_{BR}| + 1$. $|U_{BR}| \equiv_3 1$ implies $|E_\delta| \leq \frac{2}{3}|U_{BR}| + \frac{4}{3}$. $|U_{BR}| \equiv_3 2$ implies $|E_\delta| \leq \frac{2}{3}|U_{BR}| + \frac{2}{3}$, no greater than $\frac{2}{3}|U_{BR}| + \frac{4}{3} = \frac{2}{3}|U_{BR}| + |U_B| + \frac{1}{3}|U_R|$

Case 2.3 k is even, path starts with an online vertex. $|U_{BB}| = 1$, $|U_{BR}| = \frac{k}{2} - 1$. As above $|E_\delta| \leq \lceil \frac{k}{3} \rceil = \left\lceil \frac{2|U_{BR}|}{3} + \frac{2}{e} \right\rceil$ and, following a symmetric to above modulo 3 case analysis proves that $|E_\delta| \leq \frac{2}{3}|U_{BR}| + \frac{4}{3} = \frac{2}{3}|U_{BR}| + \frac{4}{3}|U_{BB}|$

□

With that relation proven we can proceed to the proof of the Theorem 6

Proof. Bounds in Equation 5.2,Equation 5.4 hold with probability $1 - e^{\Omega(n)}$, by union bound both hold with probability $1 - e^{\Omega(n)}$
From Lemma 5.2.1, Equation 5.4 we get

$$\text{OPT} \leq \left(\frac{4}{3} - \frac{2}{3e}\right) U_{BR} + \left(\frac{5}{3} - \frac{4}{3e}\right) U_{BB} + \left(1 - \frac{1}{e}\right) (U_B, U_R) + \varepsilon n$$

By choosing a small enough ε and Applying Equation 5.2 results in

$$\frac{\text{ALG}}{\text{OPT}} + \varepsilon \geq \min \left\{ \frac{1 - \frac{1}{e^2}}{\frac{5}{3} - \frac{4}{3e}}, \frac{1 - \frac{2}{e^2}}{\frac{4}{3} - \frac{2}{3e}}, \frac{1 - \frac{3}{2e}}{1 - \frac{1}{e}} \right\} = \frac{1 - \frac{2}{e^2}}{\frac{4}{3} - \frac{2}{3e}} \approx 0.670$$

□

concluding the proof

We note here that this is not the most competitive i.i.d algorithm and that there is another with a competitiveness of 0.729 which is close to the upper boundary of the competitiveness ranking in the random arrivals model. Though this isn't certain it's reasonable to assume that this model is easier than random arrivals

Chapter 6

Recourse Model

Definition 6.0.1. *Online Bipartite Matching with Recourse The Recourse model is similar to the Semi-Online model in structure, but different in function. One anticlique U is still always known, and the other, V, still arrives in an online fashion, but the goal of the algorithm is to maintain a maximal matching with as few changes to M as possible. The sum of these changes over all arriving vertices is the algorithm's Recourse, a way of measuring the algorithm like competitiveness. The goal when creating an algorithm is to minimize Recourse*

we will begin the introduction with a simple proof

Remark 6.0.1. *In the Online Edge Arrival Recourse model, where individual edges arrived in an online fashion and all vertices are visible, no algorithm has a recourse below $O\Omega(n^2)$*

Proof. The graph is a simple even path $(v_1, \ldots, v_{2k})$. First edge $(v_k, v_k{+}1)$ arrives and are matched. Then, for $i \in (1, \ldots, k-1)$, edges $(v_{k-i}, v_{k-i+1}), (v_{k+i}, v_{k+i+1})$ arrive sequentially, effectively extending the path from the front and back. Once every two edge arrivals the maximal matching changes and becomes the set of all edges except the previous matching, forcing the algorithm to change the entire matching. Thus, a total of $\Omega(n^2)$ changes is required □

Remark 6.0.2. *The same logic can be used to prove that Matching with Recourse with general vertex arrival has a lower bound of $O(n^2)$, by having vertices arrive on both sides of a odd length path.*

In contrast, in Online Bipartite Matching with Recourse all vertices of one anticlique are visible, so there is no way to extend a path by one indefinitely. This distinction is important, as

Theorem 7. *There is an algorithm, SAP, that in has a Recourse of $O(n \log^2 n)$ in the Online Bipartite Matching with Recourse model*

6.1 SAP Algorithm

the algorithm presented, as well as the proof follow the ideas from [BHR18]

Algorithm 5 SAP

on Arrival of vertex v: Augment the matching with the shortest augmenting path from v is exists

The crucial part of this proof is the proof of the following

Lemma 6.1.1 (Augmentation Lemma)**.** *$\forall h \in \mathbb{Z}$, SAP, over it's entire runtime, augments the matching with a total of no more then $\frac{4n\ln(n)}{h}$ augmenting paths longer than h*

First, we will prove SAP's correctness

Lemma 6.1.2. *SAP maintains a maximum matching*

Proof. We will conduct the proof by induction over time t. Time at $t = 0$ when no vertices have arrived forms a trivial base case with the maximal matching being $M = \emptyset$. For the induction step: Knowing

1. At time t, the matching M_t is maximal, and
2. A matching is maximal iff there are no augmenting paths

we conclude that if M_t isn't maximal at time $t + 1$ then there must be an augmenting path that didn't exist at time t, meaning that the newly arriving v must be it's starting point □

Then, we'll prove the Recourse bound using Lemma 6.1.1

Proof. We will sum all possible h augmenting path lengths by grouping them into intervals $l_i = [2^i, 2^{i+1})$
Paths in l_i contain no more than 2^{i+1} edges and there are no more than $\frac{4n\log n}{2^i}$ in each group, giving $O(n \log n)$ edges per group. Since there are $\log_2(n) + 1$ groups, total Recourse, which is no greater than the sum of all edges in all paths in all groups is no greater than $O(n \log^2 n)$ □

and finally, we will present the proof of Lemma 6.1.1 in the following sections. We will first define a Server Flow and prove several properties in the first section to prove Lemma 6.1.1 in the second section

6.1.1 Server Flow

In this subsection we will prove the Augmentation Lemma

A server flow is a fractional assignment from V to U, in which an online vertex sends flow, up to 1, to offline vertices (servers), every one of which has a unique capacity to receive flow.

Definition 6.1.1 (Server Flow)**.** *A server flow is α is a map from U to $\mathbb{R}_{\geq 0}$ such that there are non-negative reals $\{x_e | e \in E\}$ such that*

$$\forall v \in V : \sum_{u \in N(v)} x_{uv} = 1 \qquad \forall u \in U : \sum_{v \in N(U)} x_{uv} = \alpha(u)$$

we refer to these x-values as realizing the flow. Note that if $\forall u : \alpha(u) = 1$ then this is a fractional matching.
Going forward we assume $\forall v \in V : \sum_{u \in N(v)} x_{uv} \geq 1$ as otherwise no server flow is possible

Informally, a Balanced Server Flow is one where

Definition 6.1.2 (Balanced Server Flow)**.** *A Server Flow is Balanced if*

$$A(v) = \underset{u \in N(v)}{\text{argmin}} \alpha(u)$$
$$\forall v \in V \forall u \in N(v) \backslash A(v) : x_{uv} = 0$$

where $A(v)$ are called active neighbors of v

Lemma 6.1.3. *A balanced server flow exists iff $\forall v \in V : |N(v)| \geq 1$*

Proof. Proving requirement (if) simple, if an online vertex doesn't have a neighbor, it's resource cannot be received, and there is no server flow.
We will prove uniqueness by using convex programming. The program

$$\begin{aligned} \text{minimize} \sum_{u \in U} \alpha(u)^2 & \\ \text{subject to} \sum_{u \in N(v)} x_{uv} = 1 & \qquad \forall v \in V \\ \sum_{v \in N(u)} x_{uv} = \alpha(u) & \qquad \forall u \in U \end{aligned}$$

is convex, and moreover, the objective function is strictly convex (due to being a n-dimensional paraboloid) meaning that the program has a unique (with regard to α-values) optimum
Observe that the optimum is a balanced server flow: constraint ensure that it is a server flow and if it wasn't balanced there would be an online vertex v that sends non-zero flow to u, u' and $\alpha(u) < \alpha(u')$. Notice that the objective function

can be decreased by increasing $\alpha(u)$ and reducing $\alpha(u')$ until they are identical, therefore the flow must be balanced. □

We will now prove that the balanced server flow is unique (regarding x-values) using an auxiliary lemma

Lemma 6.1.4. *For a balanced server flow x_{uv} Let $\alpha_u = \sum_{v \in V} x_{uv}$ be server flow of u*
Let x'_{uv} be feasible server flow
Let $\alpha'_u = \sum_{v \in V} x'_{uv}$ be resulting server load

$$\sum_{u \in U} \alpha_u^2 \leq \sum_{u \in U} \alpha_u \alpha'_u$$

Proof. for $v \in V$ let $\mu(v) = \min_{u \in N(v)} \alpha_u$ be the minimum server load of $u \in N(v)$

$$\begin{aligned}
\sum_{u \in U} \alpha_u^2 = & \sum_{u \in U} \sum_{v \in V} x_{uv} \alpha_u = \sum_{v \in V} \sum_{u \in U} x_{uv} \alpha_u = \sum_{v \in V} \sum_{u \in U} x_{uv} \mu(v) \\
= \sum_{u \in U} x_{uv} \mu(v) = & \sum_{v \in V} \sum_{u \in U} x'_{uv} \mu(v) \leq \sum_{v \in V} \sum_{u \in U} x'_{uv} \alpha_u = \sum_{u \in U} \sum_{v \in V} x'_{uv} \alpha_u \\
= \sum_{u \in U} \alpha'_u \alpha_u &
\end{aligned}$$

with the inequality being caused by the fact that the flow x_{uv} is balanced, so $\alpha_u \geq \mu(v)$ for any edge $(u, v) \in E$ □

Lemma 6.1.5. *The server flow in Lemma 6.1.3 is unique if it exists*

Proof. Using Lemma 6.1.4 we can argue that the balanced server flow is the optimum of the linear program in the proof of Lemma 6.1.3.
We can treat a feasible solution α' as a vector, with Lemma 6.1.4 showing that $||\alpha|| \leq \alpha\alpha'$ which with non-negativity implies $||\alpha|| \leq ||\alpha'||$
Therefore, any balanced server flow is the optimum of the convex program, which in turn implies that it must be unique since the optimum is unique. □

Let α be the balanced server flow at time t and α' be the balanced server flow at time $t + 1$, after a new online vertex arrives.
$\Delta\alpha(u) = \alpha(u) - \alpha'(u)$

Lemma 6.1.6. $\forall u \in U : \Delta\alpha(s) \geq 0$

Proof. Let $U' = \{u | \Delta\alpha(u) < 0\}$
We will prove by contradiction that $U' = \emptyset$
Assume $U' \neq \emptyset$, let $\alpha^\star = \min_{u \in U'} \alpha'(u)$.

Let $U^- = \{u \in U | \alpha(u) \leq \alpha^\star\}$
Let $U^\Delta = \{u \in U | \alpha(u) > \alpha' \wedge \alpha'(u) = \alpha^\star\}$
Let $U^+ = \{u \in U | \alpha(u) > \alpha' \wedge \alpha'(u) > \alpha^\star\}$
These three partition U, $\emptyset \neq U^\Delta \subseteq U'$
Let V^Δ be the set of all $v \in V$ with an active neighbor at time t. Since each sends one unit of flow, $\sum_{u \in U} \alpha(u) \leq |V^\Delta|$
The server flow being balanced at t implies there is no edges from V^Δ to S^- at time t.
Further, there are no active edges from V^Δ to S^+ at time $t+1$
Therefore, all active edges incident to V^Δ must connect to U^Δ implying $\sum_{u \in U} \alpha'(u) \geq |V^\Delta|$.
This in turn, when combined with the previous inequality implies $\sum_{u \in U} \alpha'(u) \geq \sum_{u \in U} \alpha(u)$
This is a contradiction as, by definition of U^Δ, $\sum_{u \in U} \alpha'(u) < \sum_{u \in U} \alpha(u)$ □

Lemma 6.1.7. *when v arrives,* $\forall u \in U : \alpha(u) < \min_{u' \in N(v)} \alpha(u') \implies \Delta\alpha(u) = 0$

Or, more informally ans simplistically, arrival of v only connected to s cannot affect the flow of offline vertices whose flow is less than the flow among neighbors of s

Proof. consider the balanced server flow at time t (before inserting v)
Let $U^+ = \{u \in U | \alpha(u) \geq \min_{u' \in N(v)} \alpha(u')\}$
Ler $U^- = U \backslash U^+$ be complement of U^+
Let $V^+ = \{v \in V | N(v) \subseteq U^+\}$. Note that, before the insertion of v, there are no edges between V^+ and U^-, together with the fact that there are no active edges from V^- to U^+ because the flow is balanced implying $\sum_{u \in U} \alpha(u) = |V^-|$
After the insertion of v, by definition of U^- v has no neighbors in U^-, maintaining previous property, implying the new balanced flow α' has the property $\sum_{u \in U} \alpha'(u) \leq |V^-|$, meaning that $\sum_{u \in U} \Delta\alpha(u) \leq 0$
If, for some u_1, $\Delta\alpha(u_1) > 0$ then $\Delta\alpha(u_2) < 0$ for some other u_2, contradicting Lemma 6.1.6 □

Lemma 6.1.8.

$$\forall T \subseteq U : |\{v \in V | A(v) \subseteq T\}| \leq \sum_{u \in T} \alpha(u) \leq |\{v \in V | A(c) \cap T \neq \emptyset\}|$$

Proof. The first inequality is a direct result of the fact that every online vertex in the first set contributes exactly 1 to the sum
The second inequality is a direct result of the fact that every online vertex

contributes exactly 1 to $\sum_{u \in U} \alpha(u)$, and in the inequality every online vertex contributing anything to $\sum_{u \in T} \alpha(u)$ is counted as if contributing 1 □

Lemma 6.1.9. *Let* $\overline{\alpha} = \min_{u \in U} \alpha(u)$ *be maximal necessity,* $\overline{U} = \{u \in U | \alpha(u) = \overline{\alpha}\}$ *maximally necessary servers,* $K' = \{v \in V | A(v) \cap \overline{S} \neq \emptyset\}$ *active neighbors, then* $N(K') = \overline{U}$, $|K'| = \overline{\alpha}|\overline{U}|$

Proof. Let $K = \{v \in V | A(v) \subseteq \overline{U}\}$
Note that $K \subseteq K'$
Note that, by definition of $\overline{U}$, $K' \neq \emptyset$ and $v \in K' \implies N(v) = A(v) \subseteq \overline{U}$
Therefore $K = K'$, $N(K') = \overline{U}$.
By the previous lemma $|K'| = |K| \leq \overline{\alpha}|\overline{U}| \leq |K'|$ implying equality □

Lemma 6.1.10.

$$\overline{\alpha} = \max_{\emptyset \subset K \subseteq V} \frac{|K|}{|N(K)|}$$

and $\forall K \subseteq V : |K| = \overline{\alpha}|N(K)| \implies \forall u \in N(K) \alpha(u) = \overline{\alpha}$

Proof. Definition of server flow implies, for $K \subseteq V$ $|K| \leq \sum_{u \in N(K)} \alpha(s) \leq \overline{\alpha}|N(K)|$
implying $|K| \leq \overline{\alpha}|N(K)|$.
Let K' be defined as in above lemma. Then $\overline{\alpha} = \frac{|K'|}{N(K')} \leq \min_{\emptyset \subset K \subseteq V} \frac{|K|}{|N(K)|} \leq \overline{\alpha}$
If $|K| = \sum_{u \in N(K)} \alpha(u) = \overline{\alpha}|N(K)|$ then $\forall u \in U : \alpha(u) \leq \overline{\alpha}$ implies $\forall u \in N(K) : \alpha(u) = \overline{\alpha}$
□

subsectionReplacement Analysis We will now draw a connection between Server Flow and Matching and use the previous lemmas to prove Lemma 6.1.1

Lemma 6.1.11. $G = (U, V, E)$ *contains a matching of size* $|V|$ *iff* $\forall u \in U \alpha(u) \leq 1$

Proof. The lemma follows directly from Lemma 6.1.10: $\forall K \subseteq V : |K| \leq |N(K)|$ iff $\overline{\alpha} \leq 1$. □

Define $V_M \subseteq V$ as follows: when v arrives, V' are the vertices that arrived previously. $v \in V_M$ iff the maximum matching after v arrived is greater than the one before.
$G_M = (U, V_M, E)$ as opposed to the final graph $G = (U, V, E)$
α_M is the balanced server flow in G_M. By Lemma 6.1.5 it is unique An augmenting tail from vertex $v \in U \cup V$ is an alternating path starting at v and ending at an unmatched $u \in U$. Tail is active if all edges that aren't in the matching are active

Lemma 6.1.12 (Expansion)**.** *Assume $\alpha_M(u) = 1 - \varepsilon$ for some $u \in U, \varepsilon > 0$. There is an active augmenting tail for u of length at most $\frac{2}{e}\ln(|V_M|)$*

Proof. Notice that any offline vertex u' reachable from u by an active augmenting tail has $\alpha_M(u') \leq 1 - \varepsilon$
Let K_i be online vertices v such that there is an augmenting tail $u_0, v_0, \ldots, v_{k-1}, u_k$ where $u = u_0, v = v_j$ for some $j < i$. Do note that $v \in K_i \implies v \in K_{i+1}$.

$$|K_i| \leq \sum_{u' \in A(K_i)} \alpha_M(u') \leq \sum_{u' \in A(K_i)} (1 - \varepsilon) = |A(K_i)|(1 - \varepsilon)$$

Implying $|A(K_i)| \geq \frac{|K_i|}{1-\varepsilon}$. Assuming no active augmenting tails from u of length at least $2(i-1)$, every offline vertex in $A(K_i)$ is matched and the set of offline vertices they are matched to is K_{i+1}
Bijection $A(K_i) \leftrightarrow K_{i+1}$ is implied by the perfect matching, so $|K_{i+1}| = |A(K_i)|$ therefore $|V_M| \geq |K_{i+1}| \geq \left(\frac{1}{1-\varepsilon}\right)^i |K_1| = \left(\frac{1}{1-\varepsilon}\right)^i$. This, alongside $1 - e \leq e^{-epsilon}$ imply that $i \leq \frac{\ln|V_M|}{\ln\frac{1}{1-\varepsilon}} \leq \frac{1}{\varepsilon}\ln|V_M|$. Therefore, for any $i > \frac{1}{\varepsilon}\ln|V_M|$ there is an active augmenting tail of length at most $2(i-1)$ implying the lemma □

We can now proceed to the proof of Lemma 6.1.1. Recall the lemma statement: By, on vertex arrival, adding augmenting paths from the arriving vertex if possible, no more than $\frac{4n\ln n}{h}$ augmenting paths of length greater than h are added in total

Proof. $n = |V| \geq |V_M|$. For $h \leq 4\ln n$ this lemma holds trivially since there are at most n augmenting paths in total. We therefore assume $h > 4\ln n$ for the remainder of the proof.
Note that any augmenting path must be in G_M
Let $V' \subseteq V_M$ be the online vertices whose shortest augmenting paths are longer than $h+1$ when they are added, the lemma is equivalent to the statement $|V'| \leq \frac{4n\ln n}{h}$
For $v \in V'$ the shortest augmenting tail from each $u \in N(v)$ has length at least h, so by Lemma 6.1.12 each has $\alpha_M(u) \geq 1 - \frac{2\ln n}{h}$.
Let U' be all online vertices u that some point have $\alpha_M(u) \geq 1 - \frac{2\ln n}{h}$. By Lemma 6.1.6 U' is also the set of offline vertices u that $\alpha_M(u) \geq 1 - \frac{2\ln n}{h}$ in the final graph
By Lemma 6.1.7 $v \in V'$ implies arrival of v only increases the flow of offline vertices in U' that had a flow not below $1 - \frac{2\ln n}{h}$ before arrival.
Since G_M has a perfect matching, $\forall u \in U : \alpha_M(u) \leq \alpha(u) \leq 1$, implying the flow can only increase by no more than $2n\ln n$
Since each $v \in V$ contributes exactly 1 flow, the total number of these online vertices is $|V'| < |U'|\frac{2\ln n}{h}$
Again, each $v \in V_M$ sends 1 flow implying $n \geq |V_M| \geq (1 - \frac{2\ln n}{h})|U'| > \frac{|U'|}{2}$ since $h > 4\ln n$. This results in $|V'| < |U'|\frac{2\ln n}{h} < \frac{4n\ln n}{h}$ proving the lemma □

Chapter 7

Final Degree Model

Definition 7.0.1 (Final Degree Model)**.** *In the Final Degree Model, The algorithm has, at all times, the knowledge of the what the degree of all offline edges is in the final graph. This allows an algorithm to know how many opportunities there will be for a vertex to be matched and let it focus on the most urgent vertex.*

A natural expression of this idea is the effective degree algorithm.

Algorithm 6 Effective Degree

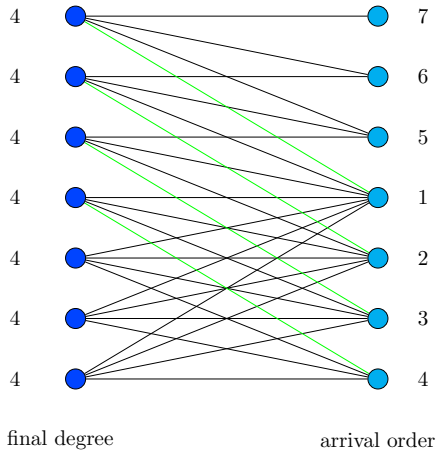
on Initialization: for each offline vertex u_i let $d_{u_i} = \deg(u_i)$
on Arrival of vertex v: N are unmatched offline neighbors of v. If $N \neq \emptyset$, match v to $\underset{u \in N}{\operatorname{argmin}}\, d_u$, $\underset{u \in N}{\forall} d_u - = 1$

Despite this algorithm seeming promising, it is only marginally more effective than a naive deterministic approach. We will show that in this model no deterministic algorithm has asymptotic competitiveness greater then 0.5

Theorem 8. *In the Final Degree model no deterministic algorithm has a competitiveness higher than* 0.5

Proof. Consider the graph $G_{\deg}$ which has $n = 2k + 1$ offline vertices with final degree $k + 1$ and is formed as follows:
$k + 1$ vertices arrive, all connected to all unmatched offline vertices.
Notice that at this point $k + 1$ vertices are matched and the remaining k have degree $k + 1$ in the current graph, meaning that they cannot be matched. All that remains is to fulfill the degree promise. This is easy as the degrees of the matched vertices are $1, 2, \ldots, k, k + 1$ meaning that all that is required is for k vertices connected to all offline vertices with unfulfilled degree promise to arrive. The size of the matching found is $k + 1$ while the graph has a perfect matching of size $n = 2k + 1$ which is achieved when the k vertices matched by the graph

are instead matched to the vertices which fulfilled their degree promise while the remaining $k+1$ vertices are matched among themselves since they form a biclique $K_{k+1,k+1}$ □



We will remark here that this model is a relaxation of the Semi-Online model, so competitiveness here isn't lower

7.1 Randomized Augmentation

As shown above, the information of a graph's final degree isn't helpful for deterministic algorithms. However, there is a good reason to believe that this might not be the case for randomized algorithms
Here, the fact that the competitiveness is only 0.5 asymptotically and in actuality is $\frac{k+1}{2k+1}$ is key.

Lemma 7.1.1. *If all vertices have different degrees then the algorithm finds a perfect matching*

Proof. Let $G_n = (V, U, E)$ be a $n \times n$ bipartite graph with with all vertices in U having different degrees. WLOG G has a perfect matching, implying that the degrees of $u \in U$ are $1, \ldots, n$. Let $u_k := u \in U | deg(u) = k$
To force the algorithm to leave n_k behind, k online vertices need to arrive, each connected to a different $u'_i \in \{u_i | i < k\}$. The problem with this is that the size of U' is $k-1$, making such a thing impossible □

A much stronger realization can be gleamed from this reasoning to force a deterministic algorithm to leave a vertex u behind, and for it to be possible for a randomized algorithm to leave that vertex behind, there must be $\deg(u)$ vertices

v such that $\deg(u) \geq \deg(v)$, which moreover won't be left behind.

This line of reasoning leads directly to the graph $G_{\deg}$ used in the proof of Theorem 8. However, this also leads to a problem for an adversary. That graph is rather dense, which isn't a situation where randomized graphs perform poorly. Indeed, despite RGA (Algorithm 3) having an expected competitiveness of $\frac{n}{2} + O(\log n)$, it is expected to find a matching of size $n(1 - \frac{1}{e})$ on a graph which is actually a subgraph of the degree counter-graph edgewise, meaning it would work at least as well here

Algorithm 7 Randomized Effective Degree

on Initialization: for each offline vertex u_i let $d_{u_i} = \deg(u_i)$
on Arrival of vertex v: N are unmatched offline neighbors of v. If $N \neq \emptyset$, let $d = \min_{u \in N} d_u$. $v = \text{random}(\{u \in N | \deg(v) = d\})$. Match v to u, $\forall_{u \in N} d_u - = 1$

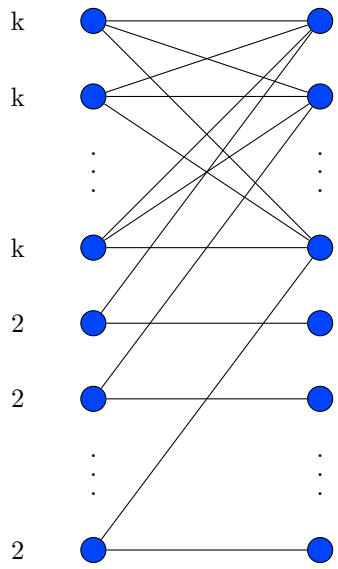
Lemma 7.1.2. *On $G_{\deg}$ from Theorem 7 Algorithm 7 has an expected performance of no less than ≈ 0.75*

Proof. We can divide the arriving online vertices into two phases: first being when the first $k+1$ vertices that make the upper k offline vertices unmatchable in the deterministic scenario, and the second phase when latter k online vertices arrive and are unmatchable.
In the first phase the arriving vertices are always matchable since they are connected to all offline vertices
In the second phase arriving vertices are connected to incrementally smaller parts of the upper k vertices
The lower $k+1$ offline and online vertices form a biclique, meaning that all matches made between the first $k+1$ online vertices and the lower $k+1$ offline vertices do not make any vertex unmatchable. Since matches are chosen randomly, we expect half of the online agents from the first phase to be matched there
In the second phase k vertices arrive connected to incrementally smaller parts of the upper k offline vertices. Using an argument symmetrical to the one in Lemma 7.1.1 we know that the matching here is perfect
In total - all online vertices in the first phase have been matched and an expected half of the vertices in the second phase has ben matched, resulting in an expected performance of 0.75 □

Note here that we have conducted Monte Carlo experiments with this exact algorithm and graph for k up to 50 and in over 1 million simulations the algorithm found a matching of average size $n - 2$ suggesting that the actual performance is much higher here

Despite these seeming advantages, this algorithm doesn't have a competitiveness above 0.5

Theorem 9. *Any algorithm that prioritizes vertices based on degree has an asymptotic competitiveness of no more than* 0.5



Proof. The case in which higher degree vertices are prioritized for matching is trivial as the counterexample is the same one from [ref-Z] and is added here for completion. In the case that the algorithm prioritized lower degree vertices like RED we construct the graph as follows: k upper offline vertices $u_1, dots, u_k$ with degree k and k lower offline vertices $u_{k+1}, dots, u_{2k}$ with degree 2. Online vertices arrive in two stages: first, k vertices $v_1, \ldots, v_k$ connected to the upper half, where v_i is also connected to $u_k + i$. These will all, except possibly the last two, be matched to $u_k + i$ as only then the effective degrees between upper and lower half equalize. This in then makes the next k online vertices unmatchable as u_{k+i} is only connected to v_{k+i} □

We note that the graph for low-priority algorithms was devised by [MY11] to fool their MPD algorithm

7.2 Performance on random graphs

as shown before, despite it's potential the Final Degree model doesn't allow for a better competitiveness. However, unlike in the Semi-Online model, the class of graphs that can force a pessimistic matching of size $\frac{n}{2}$ is very limited. For this reason, it is also worth analyzing performance in a much more forgiving, and realistic, environment.

Definition 7.2.1. *The CLV-B model is a bipartite adaptation of the CLV model for general graphs introduced by Fan Chung, Linyuan Lu, and Van Vu. It is defined by two vectors,* $p \in [0,1]^n, q \in [0,1]^m$*.An edge between* $u_j \in U, v_i \in V$ *appears with probability* $p_j q_i$*. We assume WLOG that elements of* p *are in non-increasing order.*
We denote the random graph of vectors p, q $I_{p,q}$

Do note that in this model, much like in the i.i.d model, the adversary is severely restricted and this model resembles a performance on a random graph. Adapting the RED algorithm to the CLV-B model, with consideration to the fact that there is only the expected value of any final degree and no definitive final degree, results in the following algorithm, described in [CI21]

Algorithm 8 MinPredictedDegree - MPD

on Initialization: for each offline vertex u_i let $d_{u_i} = \sum_{v=\in \vec{v_i}} v\vec{u}_i$

on Arrival of vertex v: match to $\underset{u \in U, \text{unmatched}}{\operatorname{argmin}} d_u$

Theorem 10. *MPD is optimal in CLV-B model, meaning*
for any algorithm $\mathrm{A}, t \in \mathbb{Z}_+, p \in [0,1]^n, q \in [0,1]^m$
$\Pr[\mathrm{A}(I_{p,q}) > t] \leq \Pr[\mathrm{MPD}(I_{p,q}) > t]$

the proof, as well as the algorithm itself if from [CI21] whose work concerned a slightly different model, where the algorithm has (possibly imperfect) degree predictors, but the analysis assumed that predictors follow the expected degrees, so is applicable here. It is conducted by proving, with induction, two relatively simple properties of MPD from which the theorem is proven by induction on the amount of online vertices.

Lemma 7.2.1. $p \in [0,1]^n, q \in [0,1]^m$*. For some* $i \in [n]$*,* $p' \in [0,1]^{n-1}$ *is obtained by removing entry on index* i
$\forall k$:
$\Pr[\mathrm{MPD}(I_{p,q}) \geq t] \leq \Pr[\mathrm{MPD}(I_{p',q}) \geq t-1]$

Proof. let $V = (v_1, \ldots, v_m), U = (u_1, \ldots, u_n), U' = U(u_1, \ldots, u_{i-1}, u_{i+1}, \ldots, u_n)$
For any bipartite graph G on (U, V) we will generate G' on (U', V) as follows
$\forall v \in V : N_{G'}(v) := N_G(v) \cap U' = N_G(v) \backslash \{u_i\}$
If G is distributed as $I_{p,q}$ then G' is distributed as $I_{p',q}$. Therefore, all that is necessary to prove the lemma is to show that $\mathrm{MPD}(G') \geq \mathrm{MPD}(G) - 1$, which we will to by induction on m
base case: when $m = 0$ is trivial. There are no edges so all matchings found on G, G' are of size 0
induction step: will be analyzed in cases.
Let $M_j \subseteq U$ be set of $u \in U$ matched by MPD on I after $v_1, \ldots, v_j$ arrive, M'_j defined symmetrically subset U' on I'

Case 1 MPDdoesn't match u_i when running on I
In that case the behavior of MPDon I and I' are identical, $|\text{MPD}(I)| = |\text{MPD}(I')|$, condition holds

Case 2 MPDmatches u_i to v_j when running on I
$\forall j' < j : M_{j'} = M'_{j'}$, particularly for $j' = j - 1$

Case 2.1 If v_j is unmatchable on I' then MPDon both I, I' behave identically from this point onwards, $\text{MPD}(I) = \text{MPD}(I') \cup \{(u_i, v_j)\}$, implying $|\text{MPD}(I)| = |\text{MPD}(I') \cup \{(u_i, v_j)| = |\text{MPD}(I)| + 1\}$, condition holds

Case 2.2 If v_j is matched to $u_{i'}$ on I'
Then, $|M_j| = |M'_j|$ and, by the induction hypothesis $|M_m \backslash M_j| \leq |M'_m \backslash M'_j| + 1$ (why?).
Finally $\text{MPD}(I) = |M_j| + |M_m \backslash M_j| \leq |M'_j| + |M'_m \backslash M'_j| + 1 = \text{MPD}(I') + 1$, condition holds

□

Lemma 7.2.2. *$p, p' \in [0,1]^n$ ordered, $q \in [0,1]^m$, $p \leq p'$.*
$\forall t : \Pr[\text{MPD}(I_{p,q}) \geq t] \leq \Pr[\text{MPD}(I_{p',q}) \geq t]$

In other words, if the probabilities of edges increase, then the expected size of the matching also increases

Proof. For $S \subseteq [n]$ set of indices of offline vertices, $\Pr_p[N_S]$ is the probability that, in $I_{p,q}$, the first online node's neighborhood is S, depending on p and q_1
$\Pr_p(N_S) = \prod_{i \in S} p_i q_1 \prod_{i \in [n] \backslash S} (1 - p_i q_1)$
$p(A, S)$ is vector of ordered weights of the unmatched offline vertices after running algorithm A, after the arrival of the first online node
This proof will, once again, rely on induction on m, and the case of $m = 0$ is, again, trivial since $\text{MPD}(I_{p,q}) = \text{MPD}(I_{p',q}) = \emptyset$ for the induction step:
define $(m-1)$ dimensional vector $q' = (q_2, \ldots, q_m)$

$$\Pr[\text{MPD}(I_{p,q})] \geq t = \Pr_p[N_\emptyset] \Pr[\text{MPD}(_{p,q'}) \geq t] + \sum_{\emptyset \neq S \subseteq [n]} \Pr_p[N_S] \Pr[\text{MPD}(I_{p(\vec{\text{MPD}},S),q'}) \geq t - 1] \tag{7.1}$$

let $\vec{r} = pq_1, \vec{r'} = p'q_1$, $\vec{r}_i, \vec{r'}_i$ are probabilities that v_1 is connected to u_i, u'_i in $I_{p,q}, I_{p',q}$ respectively. $p \leq p' \implies r \leq r'$. Particularly, $1 - \vec{r'}_i = (1 - s_i)(1 - \vec{r}_i)$ for some s_i,
Let X_i, Y_i independent Bernoulli variables $\Pr[X_i = 1] = \vec{r}_i$,$\Pr[Y_i = 1] = s_i$.
Let $Z_i = 1$ iff $X_i = 1 \vee Y_i = 1$ Bernoulli variable,then $\Pr[Z_i = 1] = \vec{r'}_i$
Let $N = \{i \in [n] | X_i = 1\}$,$N \supseteq N' = \{i \in [n] | Z_i = 1\}$. For any $T \subseteq [n]$.

$\Delta(S,T) := \Pr[N' = T | N = S]$

$$\Pr_{p'}[N_T] = \Pr[N' = T] = \sum_{S \subseteq T} \Pr[N = S] \Pr[N' = T | N = S] = \sum_{S \subseteq T} \Pr_{p}(N_S) \Delta(S,T)$$

note that $\sum_{T \supseteq S} \Pr[N' = T | N = S] = 1$
using Equation 7.1 we can write

$$\begin{aligned}
\Pr[\mathrm{MPD}(I_{p',q}) \geq t] &= \Pr_{p'}[N_\emptyset] \Pr[\mathrm{MPD}(I_{p',q'}) \geq t] + \\
&\quad \sum_{\emptyset \neq T \subseteq [n]} \Pr_{p'}(N_T) \Pr[\mathrm{MPD}(I_{p'(\mathrm{MP}\vec{\mathrm{D}},T),q'}) \geq t-1] \\
&= \Pr_{p'}(N_\emptyset) \Pr[\mathrm{MPD}(I_{p',q'}) \geq t] + \\
&\quad \sum_{\emptyset \neq T \subseteq [n]} \sum_{S \subseteq T} \Pr_{p}[N_s] \Delta(S,T) \Pr[\mathrm{MPD}(I_{p'(\mathrm{MP}\vec{\mathrm{D}},T),q'}) \geq t-1] \\
&= \Pr_{p}[N_\emptyset] \Delta(\emptyset,\emptyset) \Pr[\mathrm{MPD}(I_{p',q'}) \geq t] + \\
&\quad \Pr_{p}[N_\emptyset] \sum_{\emptyset \neq T \subseteq [n]} \Delta(\emptyset,T) \Pr[\mathrm{MPD}(I_{p'(\mathrm{MP}\vec{\mathrm{D}},T),q'}) \geq t-1] + \\
&\quad \sum_{\emptyset \neq S \subseteq [n]} \Pr_{p}(N_S) \sum_{T \supseteq S} \Delta(S,T) \Pr[\mathrm{MPD}(I_{p'(\mathrm{MP}\vec{\mathrm{D}},T),q'}) \geq t-1]
\end{aligned} \tag{7.2}$$

Note that, for nonempty S, $S \subseteq \implies p'(\mathrm{MPD},S) \leq p'(\mathrm{MPD},T)$
Using induction hypothesis, for $S,T : T \supseteq S \neq \emptyset$

$$\Pr[\mathrm{MPD}(I_{p'(\mathrm{MP}\vec{\mathrm{D}},T),q'}) \geq t-1] \geq \Pr[\mathrm{MPD}(I_{p(\mathrm{MP}\vec{\mathrm{D}},T),q'}) \geq t-1] \tag{7.3}$$

and

$$\Pr[\mathrm{MPD}(I_{p',q'}) \geq t] \geq \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] \tag{7.4}$$

using Lemma 7.2.1 and induction hypothesis

$$\Pr[\mathrm{MPD}(I_{p'(\mathrm{MPD},T),q'}) \geq t-1] \geq \Pr[\mathrm{MPD}(I_{p',q'}) \geq t] \geq \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] \tag{7.5}$$

using Equation 7.2,Equation 7.4,Equation 7.5 with Equation 7.1 we receive the following

$$\begin{aligned}
\Pr[\mathrm{MPD}(I_{p',q}) \geq t] &\geq \Pr_{p}[N_\emptyset] \Delta(\emptyset,\emptyset) \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] + \\
&\quad \Pr_{p}(N_\emptyset) \sum_{\emptyset \neq T \subseteq [n]} \Delta(\emptyset,T) \Pr(\mathrm{MPD}(I_{p,q'}) \geq t) + \\
&\quad \sum_{\emptyset \neq S \subseteq [n]} \Pr_{p}(N_S) \sum_{T \supseteq S} \Delta(S,T) \Pr[\mathrm{MPD}(I_{p(\mathrm{MPD},S),q'}) \geq t-1]
\end{aligned}$$

which, when combines with the fact that $\sum_{T \supseteq S} \Delta(S,T) = 1$ results in

$$\Pr[\mathrm{MPD}(I_{p',q}) \geq t] \geq \Pr_p[N_\emptyset] \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] +$$
$$\sum_{\emptyset \neq S \subseteq [n]} \Pr_p[N_S] \Pr[\mathrm{MPD}(I_{p(\mathrm{MPD},S),q'}) \geq t-1] = \Pr[\mathrm{MPD}(I_{p,q}) \geq t]$$

with the final equality being, again, from Equation 7.2 □

with these lemmas we can prove Theorem 10

Proof. We prove, once again, with induction over m, and once again the base case $m = 0$ is trivial since all algorithms return an empty matching with probability of 1
for the induction step, when inequality holds for $m-1$ online nodes:
$q' = (q_2, \ldots, q_m)$, $2^{[n]} \supseteq R_A = \{S \subseteq [n] | p(A,S) = p\}$ by induction hypothesis

$$\begin{aligned}
\Pr[A(I_{p,q}) \geq t] &= \sum_{S \in R_A} \Pr_p(N_S) \Pr[A(I_{p,q'}) \geq t] + \\
&\quad \sum_{S \in [n] \setminus R_A} \Pr_p[N_S] \Pr[A(I_{p(A,S),q'}) \geq t-1] \\
&\leq \sum_{S \in R_A} \Pr_p(N_S) \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] + \\
&\quad \sum_{S \in [n] \setminus R_A} \Pr_p[N_S] \Pr[\mathrm{MPD}(I_{p(A,S),q'}) \geq t-1]
\end{aligned}$$

$S \in R_A \implies p(A,S) \leq p(A_0,S)$. Applying Lemma 7.2.1 to the first term, Lemma 7.2.2 to the second term results in

$$\begin{aligned}
&\Pr[A(I_{p,q})] \leq \Pr_p[N_\emptyset] \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] + \\
&\quad \sum_{\emptyset \neq S \in R_A} \Pr_p[N_S] \Pr[\mathrm{MPD}(I_{p(\mathrm{MPD},S),q'}) \geq t-1] + \\
&\quad \sum_{S \subseteq [n] \setminus R_A} \Pr_p[N_S] \Pr[\mathrm{MPD}(I_{p(\mathrm{MPD},S),q'}) \geq t-1] \\
&= \Pr_p[N_\emptyset] \Pr[\mathrm{MPD}(I_{p,q'}) \geq t] + \\
&\quad \sum_{\emptyset \neq S \subseteq [n]} \Pr_p[N_S] \Pr[\mathrm{MPD}(I_{p(\mathrm{MPD},S),q'}) \geq t-1] \\
&= \Pr(\mathrm{MPD}(I_{p,q'}) \geq t)
\end{aligned}$$

□

concluding the proof

Chapter 8

Fractional Matching

We have shown that deterministic algorithm don't have a good competitiveness in the online bipartite matching problem, even if half the graph is visible or the final degrees are known.
As discussed earlier, competitiveness is a very pessimistic metric, which focuses exclusively on a worst-case scenario. When trying to find a model with good deterministic competitiveness it is reasonable then to increase the worst possible performance at the cost of reducing the best possible performance, or informally to allow an algorithm to 'hedge bets'
This is the focus of the fractional approach. A fractional matching is an assignment of numbers $w_e \in [0,1]$ to edges $e \in E$ of a graph constrained by the condition that $\forall_v \in V(G) : \sum_{e \text{ connecting } v} w_e \leq 1$. The size of a fractional matching is $\sum_{e \in E} w_e$ It is a generalization of a matching since a fractional matching with $w_e \in \{0,1\}$ is a matching

Definition 8.0.1. *In the Fractional model, an algorithm finds a fractional matching in an online fashion. All other details are identical to the Semi-Online model*

In the Fractional model a deterministic algorithm can hedge it's bets to reduce worst case scenarios. When a online vertex v arrives in the Semi-Online model it has to be matched to one of it's neighbors $N(v)$ with some of these choices ultimately reducing the size of the matching found and some preserving it. The obvious difficulty is that even an oblivious adversary can force a deterministic algorithm to make the wrong choice every time.
The strength of the Fractional model is the fact that an algorithm can instead make a compromise between good and bad choices, reducing the worst outcomes in the process.
Note that in an offline setting the sizes of a graph's matching and fractional

matching are identical. This is due to the fact that a matrix describing a fractional matching (shown later) is totally unimodular and polyhedrons $Ax \leq b$ for which A is totally unimodular and b is integer (as is the case here) are integer, meaning that their vertices are integers, implying that for any optimization direction an integer vector is an optimum

Algorithm 9 Waterlevel

on Initialization: create water level of all offline vertices $\forall u \in U : d(u) = \sum_{v \in V, \text{visible}} w_{u,v}$

on Arrival of vertex v: Increase w of edges towards neighbors with the lowest water level, to maximize $\min_{u \in N(v)} d(u)$ until 1 unit has been alloted or all neighbors have a water level of 1

Theorem 11. *The Waterlevel Algorithm 9 has a competitiveness of* $1 - \frac{1}{e}$

We note that this both the algorithm and it's competitiveness are well-known. Our analysis will follow the Cornell lecture CS-6820 on Online bipartite matching algorithms.

Proof. The basis of this proof is the primal-dual method. A bipartite fractional matching is represented by the following primal-dual program. $l(v)$ denotes the new water level among neighbors of v after it arrives and is matched

$$\begin{aligned}
&\text{maximize} \sum_{(u,v)\in E} x_{uv} && \text{minimize} && \sum_{u\in U} \alpha_u + \sum_{v \in V} \beta_v \\
&\text{subject to} \forall v \in V : \sum_{u \in N(v)} x_{uv} \leq 1 && \text{subject to} && \forall u, v \in E : \alpha_u + \beta_v \geq 1 \\
&\qquad \forall u \in U : \sum_{v \in N(u)} x_{uv} \leq 1 && && \forall (u,v) \in E : x_{uv} \geq 0 \\
&\qquad \forall (u,v) \in E : x_{uv} \geq 0
\end{aligned}$$

We define a dual solution as follows

$$\begin{aligned}
\alpha_u &= g(d(u)) \\
\beta_v &= 1 - g(l(v)) \\
g(y) &= \frac{e^y - 1}{e - 1}
\end{aligned}$$

which we will prove is feasible and decreases at a certain rate as the algorithm progresses. The fact that any dual solution is a bound on any primal solution will be the basis of this proof
g has several properties

1. it is increasing
2. $g(0) = 0$
3. $g(1) = 1$
4. $1 - g(y) + g'(y) = \frac{e}{e-1}$

Observe that the defined dual solution is feasible since after the arrival of v. $u, v \in E \implies d(u) \geq l(v)$. By property 1

$$\alpha_u + \beta_v = g(d(u)) + 1 - g(l(v)) \geq 1$$

verifying dual feasibility

Lemma 8.0.1.

$$\frac{e}{e-1} \sum_{(u,v)\in E} x_{uv} \geq \sum_{u\in U} \alpha_u + \sum_{v\in V} \beta_v$$

Proof. Define β' as follows: $n_v(t)$ is the amount of $u \in N(v)$ such that the inequality $d(u) \leq t$ holds when v arrives.
$\int_0^{l(v)} n_v(t)dt = 1$ if $l(v) < 1$
$\beta'_j = \int_0^{l(v)} (1 - g(t))n_v(t)dt$
Note that $\beta'_j \geq \beta_j$ because, $l(v) = 1$ is implied by $\beta_v = 0$ and $l(v) < 1$ is implied by $1 - g(t)$ being a decreasing function and therefore

$$\beta'_v = \int_0^{l(v)} (1 - g(t))n_v(t)dt > (1 - g(l(v))) \int_0^{l(v)} n_v(t)dt = 1 - g(l(v)) = \beta_v$$

proves the claim.
We now analyze the amount by which the dual objective function changes by the arrival of vertex v. In this situation deg will denote the degree before the

arrival of v. This amount is

$$\begin{aligned}
&\beta_v + \sum_{u \in N(v)} g(l(v)) - g(\deg(u)) \\
=&1 - g(l(v)) + \sum_{u \in N(v)} \int_{\deg(u)}^{l(v)} g'(t)dt \\
=&1 - g(l(v)) + \int_{\deg(u)}^{l(v)} n_v(t) g'(t)dt \\
\leq& \int_{0}^{l(v)} \left(1 - g(t) + g'(t)\right) n_v(t)dt \\
=&\frac{e}{e-1} \int_{0}^{l(v)} n_v(t)dt \\
=&\frac{e}{e-1} \sum_{u \in N(v)} x_{uv}
\end{aligned}$$

which is no more than $\frac{e}{e-1}$ times the increase in the primal objective, proving the inequality □

With Lemma 8.0.1 proven the proof is trivial. By weak duality the dual objective is an upper bound on the size of the fractional matching, implying Theorem 11 □

Chapter 9

Other Models

9.1 Delay

Definition 9.1.1 (Delay model)**.** *The delay model an enhanced version of the Semi-Online model, where on vertex arrival it doesn't have to be matched immediately, but only after a delay of $f(n)$ time - $f(n)$ arriving online vertices*

This model isn't well-studied as despite the fact that delay (sometimes referred to as buffer or lookahead) is sometimes utilized, it isn't useful in the case of Online Bipartite Matching

Theorem 12. *For delay no greater than $n/2$ any deterministic online algorithm has competitiveness of* 0.5

Proof. Consider the following graph G: There are $k = n/2$ groups $\{Z_i | i \in [k]\}$. of two offline vertices
First, k vertices z_i^D arrive, each connected to both vertices of Z_i.
As z_k^D arrives the cache is full, and z_1^D has to be matched. Let the offline vertex it was matched to be z_1^B. Next, z_1^C arrives, only connected to z_1^B, which is unmatchable. Let z_1^A be the unmatched offline vertex.
After z_1^A has arrived z_2^D has to be matched. The procedure above is repeated until z_k^C has arrived.
The perfect matching of G connects all z_i^A, z_i^D and z_i^B, z_i^C pairs, while the algorithm connects only z_i^B, z_i^D pairs, making the competitiveness 0.5. The final graph is illustrated in the figure below. □

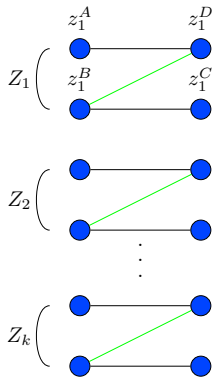


Figure 9.1: Worst case scenario for a deterministic cache algorithm

Theorem 13. *For any $f(n) = o(n)$ delay, no algorithm has asymptotic competitiveness better than $1 - \frac{1}{e}$*

Proof. For ALG construct the adversary graph in a manner similar to the one above, with $f(n)$ copies of G_A, a graph on which ALG has a performance of $1 - \frac{1}{e}$. Notice that the size of individual G_A is $\frac{n}{f(n)}$ and since $\lim_{n\to\infty} \frac{n}{f(n)} \to \infty$, the size of individual G_A instances is growing with n, leading to ALG being forced to solve isolated instances of size growing to infinity, yielding the stated competitiveness. □

9.2 Corrections

Theorem 7 proves that there is an algorithm that maintains a maximal matching with amortized $O(\log^2 n)$ corrections per arriving vertex. This produces a question: what would be the competitiveness of an algorithm that has a strict correction budget.

Definition 9.2.1 (Correction Model)**.** *In the correction model, every turn, the algorithm can remove up to a constant d edges from the formed matching. There are no restrictions on adding edges*

An algorithm that improves the matching by changing some d edges for each arrival, for some constant d, can be derived from a known lemma proven by Hopcroft and Karp in 1973. We note that others may have described a very similar model earlier, but our analysis is independent.

Lemma 9.2.1. *for a matching M, if all augmenting paths are of length no less than $2k - 1$, then $\frac{|M|}{|M^\star|} \geq \frac{k-1}{k}$*

we will follow the proof from [PFM19]

Proof. Consider that graph $G' = (U, V, E')$ where E' is the symmetric difference between some matching M and $M^\star$, which is $(M \backslash M^\star) \cup (M^\star \backslash M)$, which in turn is the difference between M and $M^\star$. Vertices in G' have a degree of either $0, 1$ or 2. Vertices which are either isolated or in cycles (which must be of even length) in G' have the same amount of edges adjacent from $M, M^\star$. Augmenting paths, of which there are no more than $|M^\star| - |M|$, must account for the difference in cardinality.

When the difference in cardinality is greatest, G' contains only augmenting paths, of which each has at least $k-1$ edges in M and k edges in $M^\star$. Let P be a path. Then $\frac{|M \cap P|}{|M^\star \cap P|} \geq \frac{k-1}{k}$ which can be generalized ove all paths to $\frac{|M|}{|M^\star|} \geq \frac{k-1}{k}$ □

From here we can construct the algorithm. We will call an augmenting path short if it is of length no greater than $2d + 1$, meaning it has d edges in the matching. Notice that an augmenting path of length $2k - 1$ only has $k - 1$ edges from the matching. We will prove the competitiveness with the above lemma,

Algorithm 10 d-augmenting correction

on Arrival of vertex v: if a short augmenting path P (starting at v) is created, $N = M \cap P$, $N' = P \backslash N$ for current matching M, $M := N' \cup M \backslash N$. otherwise do nothing

combined with a simple invariant: after an arriving vertex is handled, there are no short augmenting paths.
To create a short augmenting path, a vertex needs to be added to the right endpoint of a non-augmenting alternating path or an augmenting path must be separated. The arrival of v might fulfill the first conditions, but the algorithm subsequently switches one of these augmenting paths, matching v. The first condition never occurs since switching an augmenting path doesn't split it. Therefore, d-augmenting correction has a competitiveness of $\frac{d}{d+1}$

Chapter 10

Conclusions

Recall the results we have proven

1. In the simplest Semi-Online model, Ranking has a competitiveness of $1 - \frac{1}{e} = 0.632$ and is optimal
2. If vertices arrive in a random order, Ranking has a competitiveness above 0.696
3. If vertices arrive from a known distribution, a deterministic algorithm (TSM) has a competitiveness of 0.702
4. Even a substantial delay in decision time doesn't increase competitiveness
5. Using the knowledge of final degrees reduces competitiveness
6. In a more random graph the usage of final degrees leads to an optimal algorithm (MPD)
7. A high recourse cannot be forced
8. Waterlevel fractional matching has a competitiveness of $1 - \frac{1}{e} = 0.632$
9. A deterministic algorithm, even allowed to make small corrections, has a good competitiveness.
10. none of the mentioned models allow for a perfect algorithm, which always finds a perfect matching

In terms of the distinction between algorithm enhancement and adversary restriction, Random Arrival is a Restricted model, Final Degree, Fractional and Delay are Enhanced models and CLV-B and i.i.d are both. Recourse is fundamentally different as the change from Semi-Online is in the goal itself.

An interesting trend is apparent in the best known algorithms, in that, speaking broadly, randomized algorithms are better in all models that aren't enhanced

and Restricted at the same time, with Fractional being the outlier. A broad observation can be made that randomness is used to guard against an oblivious adversary forcing a worst-case scenario and additional capabilities, like knowledge of final degrees, is not enough to prevent it. Meanwhile, when the adversary cannot set the edges, deterministic algorithms become viable, or rather randomization loses value.

When it comes to algorithm enhancement, the difference between the non-effective (Final Degree, Delay) and the effective (Fractional, Corrections), ie the ones that are strictly easier than Semi-Online, is rather obvious - the former provides additional information while the latter changes the matching process. The conclusion is that, under the assumption of adversarial arrival, the setting is flexible enough that the additional information doesn't help the algorithm

When it comes to corrections, we can see that, at least in the case of one sided vertex arrivals, this is a powerful capability. Specifically, the combination of the visibility of one anticlique and vertex arrival make forcing many correction difficult.

Lastly, we can generalize criteria for a 'perfect' algorithm. We know that a matching is perfect iff no augmenting paths. As such we can confidently say that any model in which information on augmenting paths in the final graph isn't present doesn't allow for the existence of a perfect algorithm. A weaker argument regards time complexity. The fastest offline algorithm for bipartite matching has a time complexity of $O(|E|\sqrt{|V|})$ and we can expect that any model in which an online algorithm can run strictly faster (bearing in mind modifications such as final degree knowledge must be accounted for) doesn't allow for a perfect matching, the assumption being, of course, that the Hopcroft-Karp algorithm is optimal. A stronger version of this argument is to check whether the online algorithm can run in $o(|E|)$ time as an algorithm would not be able to take all edges into account if that was the case. However, this is an easy threshold to cross since even ranking runs in $O(|V|^2)$ time and $|E| \leq |V|^2$ by definition

Bibliography

[Kur70] Thomas G. Kurtz. "Solutions of Ordinary Differential Equations as Limits of Pure Jump Markov Processes". In: *Journal of Applied Probability* 7.1 (1970), pp. 49–58. ISSN: 00219002. URL: http://www.jstor.org/stable/3212147 (visited on 08/31/2026).

[KVV90] R. M. Karp, U. V. Vazirani, and V. V. Vazirani. "An optimal algorithm for on-line bipartite matching". In: *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*. STOC '90. Baltimore, Maryland, USA: Association for Computing Machinery, 1990, pp. 352–358. ISBN: 0897913612. DOI: 10.1145/100216.100262. URL: https://doi.org/10.1145/100216.100262.

[BM08] Benjamin Birnbaum and Claire Mathieu. "On-line bipartite matching made simple". In: *SIGACT News* 39.1 (Mar. 2008), pp. 80–87. ISSN: 0163-5700. DOI: 10.1145/1360443.1360462. URL: https://doi.org/10.1145/1360443.1360462.

[GM08] Gagan Goel and Aranyak Mehta. "Online budgeted matching in random input models with applications to Adwords". In: *Proceedings of the Nineteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '08. San Francisco, California: Society for Industrial and Applied Mathematics, 2008, pp. 982–991.

[Fel+09] Jon Feldman et al. "Online Stochastic Matching: Beating 1-1/e". In: *2009 50th Annual IEEE Symposium on Foundations of Computer Science* (2009), pp. 117–126. URL: https://api.semanticscholar.org/CorpusID:1002166.

[MY11] Mohammad Mahdian and Qiqi Yan. "Online bipartite matching with random arrivals: an approach based on strongly factor-revealing LPs". In: *Proceedings of the Forty-Third Annual ACM Symposium on Theory of Computing*. STOC '11. San Jose, California, USA: Association for Computing Machinery, 2011, pp. 597–606. ISBN: 9781450306911. DOI: 10.1145/1993636.1993716. URL: https://doi.org/10.1145/1993636.1993716.

[BHR18] Aaron Bernstein, Jacob Holm, and Eva Rotenberg. "Online bipartite matching with amortized O(log2 n) replacements". In: *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*. SODA '18. New Orleans, Louisiana: Society for Industrial and Applied Mathematics, 2018, pp. 947–959. ISBN: 9781611975031.

[PFM19] Alex Pothen, S. M. Ferdous, and Fredrik Manne. "Approximation algorithms in combinatorial scientific computing". In: *Acta Numerica* 28 (2019), pp. 541–633. DOI: 10.1017/S0962492919000035.

[CI21] Justin Y. Chen and Piotr Indyk. "Online Bipartite Matching with Predicted Degrees". In: *CoRR* abs/2110.11439 (2021). arXiv: 2110.11439. URL: https://arxiv.org/abs/2110.11439.